A benchmark of cell-type annotation methods for single-cell data: cost, not accuracy, distinguishes them

Ian Driver*

Abstract

Cell-type annotation by reference mapping is one of the most repeated operations in single-cell analysis, yet comparisons report accuracy far more often than the time and memory that decide what a scientist can actually run. We benchmarked **thirteen methods**, classical through foundation-model, across **eight splits of seven datasets** (8–151 cell types) on commodity hardware, and on Open Problems `label_projection`, whose datasets and metrics we did not choose. Accuracy among the leading methods is tightly clustered — the top four span **0.008** — while their inference cost differs by two orders of magnitude and their peak memory by 2.5×. No method leads everywhere, and no ordering is stable: it inverts with reference size, feature budget and label granularity. Because annotation is this cheap, a query can be labelled several times rather than once. We ran three procedures that do so, all using **actinn-jax**, a JAX reimplementation of ACTINN that caches a trained reference and labels a query in under a second. A 798-type human reference assigns a tissue, and a tissue-specific reference then relabels the same cells, raising cross-study liver from 0.23 exact / 0.58 ontology to 0.72 / 0.86. A pretrained pan-human annotator was distilled into an actinn-jax reference that matches its accuracy and labels six to nine times faster, built from unlabelled data without a GPU. Running three broad references over one query and grouping cells by whether the three agree separates reliable calls from unreliable ones: in liver, lung and brain every reference scores far higher where all three agree, though the consensus label beats no single reference. Code and pre-trained references are available at github.com/iandriver/actinn-jax, github.com/iandriver/actinn-jax-benchmark and doi:10.5281/zenodo.21688150.

Independent Researcher, Detroit, MI, USA
* Correspondence: driver.ian@gmail.com

1 Introduction

Annotating cell types by mapping to a labeled reference is one of the most frequently run operations in single-cell RNA-seq. The method landscape is large, but published comparisons ([Abdelaal 2019], [Fu 2024]) emphasize accuracy and rarely report the axis that decides what a working scientist actually runs on their own machine: **wall-clock time and memory on commodity hardware**, without a GPU. Foundation models (scGPT, Geneformer, scPRINT) push accuracy in some settings but need GPUs and minutes-to-hours, and used *zero-shot* they underperform far simpler alternatives: scPRINT's zero-shot labels score 0.201 ontology concordance on the lung split against 0.917 for a reference-trained model on the same cells (§3.6), and an independent evaluation places Geneformer's and scGPT's zero-shot embeddings behind scVI, Harmony and plain highly-variable-gene selection on clustering and batch correction [Kedzierska 2025].

We ask a practical question: **for a given annotation job on commodity hardware, which method should you actually run, and what does the surrounding workflow look like?**

The leading methods now annotate quickly and accurately and give a usable signal on unknown cells, so the shortage is not another leaderboard but guidance on fit-for-purpose — accuracy per second, accuracy per byte, and how each behaves as the reference grows. We answer with a neutral 13-method benchmark plus a set of demonstrated workflows, held to a single standard: every method runs in its own environment through the same harness, on the same splits, scored by the same metrics, and reported as measured — including where actinn-jax is not the leader.

How the comparison was constrained. A benchmark written by a method's own author has a well-known failure mode: the datasets, baselines and metrics that flatter the method are the ones that get reported. Every method here runs on **every** dataset, so none appears on a favorable subset, and the two methods outside that matrix are named with reasons (§2.2). The panel was chosen to include the baselines most likely to beat a small MLP — a carefully tuned linear pipeline, scTOP, ProtoCloud — and each of them does beat it somewhere; our own classical tier is left untuned while the linear baseline is tuned, which is the unfavorable direction (§5). Metrics are fixed across all methods and reported in full. And the **external validation is not ours to design**: Open Problems `label_projection` (§3.7) sets the datasets, the metrics and the ranking, and we did not choose any of them. Where results run against this method's interests — ProtoCloud overtaking actinn-jax at atlas scale, a memory advantage that stays a fixed factor instead of growing with the data, a better-resourced broad annotator, a correlation method leading the finest-grained split — they are reported in those terms.

On adding another method to a crowded field. We are not concluding that the new method wins. On accuracy it sits inside a four-way cluster it does not lead; the panel was assembled to include the baselines most likely to beat it, and they do. What earns it a place is narrower: the original ACTINN no longer installs or runs on current Python and ML environments, and the reimplementation's flat, sub-second, memory-bounded inference over a cached reference is what makes the multi-stage workflow of §3.4 practical on a laptop. A field already full of working methods (xkcd 927) is a good reason to be specific about what a new entry is *for* — here, a cost profile that lets several models be chained — rather than to claim it supersedes what exists. Where a better-resourced model already exists, the productive move is to build on it (§3.4).

Contributions.

1. A modern, dependency-light (no TensorFlow) JAX reimplementation of ACTINN with sparse preprocessing, chunked atlas-scale prediction, and a cached reference model.
2. A neutral **13-method × 6-dataset** benchmark of accuracy, speed, and memory on Apple Silicon, with a rejection/abstain analysis and a **sweep across reference sizes up to full atlas scale**, which is what shows that the ranking is a function of how much reference data each method is given. The panel deliberately includes the baselines most likely to beat a small MLP — a tuned linear pipeline, scTOP, and ProtoCloud — and the results are reported as measured (§3.1, §5). The sweep is also what sets actinn-jax's minibatch policy: at ACTINN's fixed batch of 128, a 47k-cell reference costs ~24k tiny update steps, so actinn-jax scales the batch with the reference above ~12.8k cells and keeps 128 below it. Measured fit and predict curves are in §3.3.
3. **Demonstrated end-to-end workflows** for jobs where cost is the binding constraint, all CPU: annotate an unknown human dataset from a **bundled census-wide ~800-type reference** with a calibrated abstain and no training; **tissue-aware refinement** (halves spurious cross-tissue labels on a liver query); a **broad→focused hand-off** to a small focused reference (cross-study liver 0.23/0.58 → 0.72/0.86); within-cell-type resolution (hepatocyte zonation); and a **one-call novel-cell-type screen** (recovers a withheld pulmonary ionocyte population

and its marker ASCL3). We are precise about what is not ours: ProtoCloud offers uncertainty, attribution and a retraining-based refine, and Pan-human Azimuth is a pretrained hierarchical pan-human annotator with trained abstention, so the broad pass itself is not a distinguishing feature — we distill that model into a broad-pass reference rather than compete with it. What is distinct is **refinement into a label set no pretrained model carries** — the user's own focused reference, and states below a fixed typology's leaves. The same route builds a **pan-mouse reference** (453 types / 85 tissues; 0.638 ontology concordance on two withheld datasets) in 17 s of CPU, using Cell Ontology lineage in place of a foundation-model embedding. On the human corpus where it was validated it beats having no hierarchy at all (0.616 vs 0.547) and a random grouping of the same sizes (0.539), and it needs no GPU at any stage.

4. An **independent external validation** on Open Problems `label_projection`, with a controlled same-hardware speed/memory comparison, and a set of cheap ablations that characterize *when* the gene-space model gains or loses to heavier methods (input standardization; a reference-guided gene budget; a foundation-model negative control).

2 Methods

2.1 actinn-jax

actinn-jax reimplements ACTINN ([Ma & Pellegrini 2020]) — a 4-layer fully-connected network (100/50/25 hidden units, ReLU, softmax) trained with Adam — in JAX/optax ([Bradbury 2018]), replacing the original's TensorFlow-1.x graph/session code. Key engineering:

- **Sparse-aware preprocessing**: the count matrix is never densified in full. CP10k+log2 normalization and the expression / coefficient-of-variation gene filter run on the sparse matrix; only the selected-gene columns of each minibatch are made dense.
- **Cached reference model**: train once, write the fitted model to disk, reload it and map many queries — the amortized cost of repeated annotation against a fixed reference drops to the inference cost alone.
- **Chunked prediction** for atlas-scale queries, and raw counts are picked up automatically from the slot where CELLxGENE files conventionally store them.
- **Optional reference-fit standardization**: z-score each selected gene by the reference's frozen mean and standard deviation and apply that to the query — a cheap domain alignment (§3.7). Off by default; when on, the scaler is saved alongside the model and travels with it, and models saved without it still load.

The reimplementation reproduces the original's accuracy to within repeat noise and, unlike the TensorFlow-1.x original, **installs on current Python and ML environments**. Because the two are the same model at equal accuracy, we do not carry the original as a separate benchmark row: the change is one of installability and speed, not of accuracy.

2.2 Benchmarked methods

method	source	tier	engine	rejection	runtime
actinn-jax	[Ma & Pellegrini 2020]	classical	JAX MLP	confidence threshold	JAX

method	source	tier	engine	rejection	runtime
SVM	[Pedregosa 2011]	classical	linear SVM (SGD)	—	scikit-learn
kNN	[Pedregosa 2011]	classical	k-nearest neighbors	—	scikit-learn
CellTypist	[Domínguez Conde 2022]	linear	L2 logistic regression	prob threshold	scikit-learn
linear-anova-pca	[Pedregosa 2011] †	linear	normalize → ANOVA → PCA(220) → logreg	prob	scikit-learn
scTOP	[Yampolskaya 2023]	parameter-free	rank z-score class-average projection	—	NumPy
SingleR	[Aran 2019]	correlation	Spearman + fine-tuning	—	R
scmap-cluster	[Kiselev 2018]	correlation	centroid cosine	yes (unassigned)	R
scANVI	[Xu 2021]	deep	scVI semi-supervised VAE	prob	scVI
scArches	[Lotfollahi 2022]	deep	scANVI reference surgery	prob	scVI
ProtoCloud	[Guo & Ding 2026]	deep	prototype VAE + LRP attribution	ambiguity flag	PyTorch
scPRINT	[Kalfon 2025]	foundation	pretrained transformer, zero-shot	—	PyTorch
Pan-human Azimuth	[Sarkar et al. 2026]	pretrained	8-level hierarchical NN, fixed 382-leaf typology	trained `Unassigned`	TensorFlow

Table 1. The benchmarked methods: model family (*tier*), the engine each one actually runs, whether it can decline to call a cell (*rejection*), and the runtime stack it was run on; each has its own pinned environment (§2.5). Bold marks the methods added to this panel here.

† `linear-anova-pca` has no method paper: it is a baseline assembled here from scikit-learn components, tuned deliberately to be the strongest simple competitor we could build (§1).

Eleven of the thirteen are trained on each dataset's own reference and run the full accuracy/cost matrix of §3.1 on all eight splits, so none is compared on a favorable subset. The remaining two are **pretrained annotators with fixed label vocabularies**, which changes what can be measured rather than excusing them from measurement:

- **scPRINT** is a zero-shot foundation model over a fixed **Cell Ontology (CL)** vocabulary — CL is the standard controlled vocabulary of cell types, used throughout as the common ground between datasets that name the same cell differently. scPRINT cannot emit labels outside that set, and skips symbol-keyed datasets.
- **Pan-human Azimuth** ([Sarkar et al. 2026]) is a supervised hierarchical classifier over a single organism-wide typology — 8 levels, 382 leaves, ~7M parameters, a fixed 5,055-gene panel, trained on 9.7M curated cells, with abstention *learned rather than thresholded* — the

model is trained to answer `Unassigned`, instead of having a probability cutoff applied to its output afterwards. It is the closest published counterpart to this paper's broad pass, and the teacher we distill in §3.4.

Neither predicts into the dataset's label strings, so **exact-match accuracy against those strings is a vocabulary artifact, not an accuracy signal** — `regulatory T cell` → `Treg cell` and `non-classical monocyte` → `CD16 monocyte` are correct answers that score zero. Both are therefore scored **ontology-only** (§2.4) and reported in their own block in §3.1, on the three datasets where reference and query both carry Cell Ontology ids. They appear again in §3.6 (foundation-model zero-shot) and §3.4 (Azimuth as broad pass, and as distillation teacher).

scPred is not included: it is unmaintained, and no longer installs against current releases of the harmony dependency it builds on.

2.3 Datasets

dataset	source	split	tissue	#types	genes	notes
lung_intra	[Travaglini 2020]	within-dataset	lung (Krasnow)	46	Ensembl	300 cells/type ref
lung_cross	[Sikkema 2023] → [Travaglini 2020]	cross-dataset	lung (HLCA → Krasnow)	46	Ensembl	different lab/protocol
liver_intra	[Edgar 2026]	within-dataset	liver (HLiCA)	36	Ensembl	150 cells/type
liver_cross	[Edgar 2026]	cross-**study**	liver (HLiCA)	34	Ensembl	train 6 studies → test withheld study
blood_gut_intra	[Alegbe 2026]	within-dataset	blood + gut (IBDverse)	86	Ensembl	high cardinality; no CL ids
pbmc	[10x Genomics 2016]	within-dataset	PBMC (pbmc3k)	8	symbols	small-n; scPRINT skips (symbols)
brain_intra	[Jorstad 2023]	within-dataset	brain, MTG (Allen)	24	Ensembl	single nuclei; Allen subclasses
brain_cluster	[Jorstad 2023]	within-dataset	brain, MTG (Allen)	151	Ensembl	same nuclei, CCN cell sets

Table 2. The eight benchmark splits, over seven datasets — brain appears twice because the same nuclei are scored at two levels of one taxonomy. *split* separates within-dataset holdouts from cross-dataset and cross-study transfer; *genes* records whether the matrix is keyed by Ensembl ids or by gene symbols, since a method or reference built on one convention cannot read the other without remapping.

The blood+gut set is a subsample of **IBDverse** (Wellcome Sanger Institute; blood, terminal ileum and rectum from 421 individuals), included here because 86 fine-grained labels make it the high-cardinality stress case for the panel. It carries no Cell Ontology ids, so it contributes to accuracy and macro-F1 but to no ontology-scored result.

The brain set is single nuclei from the Allen Institute's human **middle temporal gyrus** taxonomy [Jorstad 2023], the same query used in the multi-reference agreement experiment of §3.4. It enters twice, at two levels of that taxonomy: `Subclass` (24 types, ~300 cells each), the standard annotation level, and `Cluster` (151 types, ~100 cells each), whose members are the cell sets `CCN201908210` enumerates. Only the 10x 3′ v3 nuclei are kept (141,782 of 156,285); the remaining Smart-seq v4 nuclei would put a full-length protocol on one side of a within-dataset split.

Both levels are scored by exact match alone. The Cell Ontology collapses the five intratelencephalic subclasses (`L2/3 IT`, `L4 IT`, `L5 IT`, `L6 IT`, `L6 IT Car3`) into one term and merges `Lamp5`/`Lamp5_Lhx6` and `Sst`/`Sst Chodl`, leaving **18 terms for the 24 subclasses** and none for the 151 clusters, so an ontology-aware score would credit a method for confusing cortical layers. Brain therefore contributes accuracy and macro-F1 and no ontology-scored result, as blood+gut does for a different reason. §3.4 returns to this gap.

The set spans the three generalization regimes (within-dataset, across datasets, across studies), more than an order of magnitude in cell-type count (8→151), five tissues (lung, liver, blood, gut, brain), whole cells and single nuclei, and both gene-ID conventions.

2.4 Metrics

Per (dataset, method, repeat): **accuracy**; **macro-F1**, unweighted over classes so rare types count as much as common ones; **ontology-aware concordance**; **fit time**, **predict time**, and **peak memory** (peak RSS, resident set size — the maximum physical memory the process held).

Ontology-aware concordance credits a call that is the same node, an ancestor, or a descendant of the truth in the Cell Ontology. It exists because cell-type vocabularies disagree about granularity more than about biology: `periportal region hepatocyte` against `Hepatocyte` is a right-lineage-wrong-depth call. Exact match scores it as a total failure, though the lineage is correct and only the depth is wrong. It is the **only** metric that compares methods with different label vocabularies (§2.2), and it is reported only where both reference and query carry CL ids — so not on `blood_gut_intra`, and not on the symbol-keyed `pbmc`.

Concordance depends on the ontology release, since ancestor sets change between them. All numbers here use the **Cell Ontology basic release of 2026-06-08** (sha256 73996c63…), resolved once and reused by every scoring pass. Scoring lung_cross against a later-fetched release reproduces the splits exactly (14,390 reference / 13,550 query cells) and still moves concordance by 0.003, with no model changing. That release, its checksum, and pinned versions for all six benchmark environments are recorded as lockfiles in the benchmark repository, which also carries a command that reports any drift from them.

Exact-match accuracy is reported for the ten reference-trained methods and, on `lung_cross`, flagged: reference and query use different label vocabularies there, so its exact-match accuracy is a vocabulary artifact and the ranking, not the level, is what transfers.

2.5 Harness, isolation, and hardware

Each method runs in **its own virtual environment, as a separate process**, because the dependency sets are mutually unsatisfiable — scVI-based methods, TensorFlow/Keras (Pan-human Azimuth), R (SingleR, scmap) and the JAX core cannot coexist in one interpreter. One driver builds the reference/query split **once per dataset** from a fixed seed and hands the identical pair to every method, so no method sees a different split. **repeats = 3**; scPRINT and Pan-human Azimuth run once each, being deterministic given a fixed query and dominated by a single forward pass.

Resource accounting is per subprocess: fit and predict are timed separately and peak memory is sampled by a monitor thread in the child, so one method's footprint cannot be attributed to another.

Hardware: Apple Silicon (M-series), CPU for the classical, linear, correlation and pretrained tiers and for actinn-jax; Apple MPS for the deep and foundation tiers. No discrete GPU, no cloud — the regime the benchmark is about. Math-library threads are left uncapped, so wall-clock is what a practitioner sees; an optional thread cap exists for stricter reproducibility. §3.7 repeats the comparison on one cloud CPU instance, where methods compete for cores; §2.10 explains how that is handled.

2.6 Building a broad reference

The bundled broad-pass references (§3.4) are built once, offline, by a three-stage pipeline driven by one command in the benchmark repository.

Sampling. Every primary cell for one organism in the CELLxGENE Census [CZI Census 2025] — primary meaning the copy the census designates canonical, so cells deposited in more than one study are counted once — stratified by cell type at a fixed cap per type: 40 for `broad_human_v1`, 60 for the later human and mouse pulls. Types with fewer than 12 cells are dropped, as are labels that carry no information ("unknown", "native cell", "eukaryotic cell", "animal cell"). The pull is checkpointed every ten datasets and retried, since transient object-store read errors are routine at this scale. Genes are carried under both Ensembl ids and symbols, because a symbol-keyed consumer cannot use an Ensembl-only pull. The census "stable" alias is a moving pointer, so the release it resolved to is recorded; every reference here comes from **census 2025-11-08**.

Coarse hierarchy — two routes. The model is a coarse classifier over groups plus one fine classifier per group, and the grouping comes from either:

1. **Foundation-model embeddings.** scPRINT (medium-v1.5) embeds the reference once in 4,000-cell chunks; per-cell-type centroids are Ward-clustered into roughly $\sqrt{}$(number of types) groups, with a floor of eight. scPRINT's QC drops cells, so the embedding is not positionally aligned to the input and the label is carried through the model with the vectors. This is the only step that wants a GPU/MPS.
2. **Cell Ontology lineage.** Each cell type is described by a binary indicator over the CL terms it descends from; terms appearing in fewer than two types, or in all of them, are dropped as uninformative; the same Ward clustering is applied to those vectors. Free, deterministic, and species-independent — which is what makes the pan-mouse reference possible, since the distillation teacher is human-only and scPRINT's mouse support is untested here. §3.4 scores this route against no hierarchy and against a random grouping.

Training and calibration. A 4,000-gene panel of **highly variable genes (HVGs)** — the genes

that differ most across cells, and so carry most of the signal for telling types apart — is selected on the reference; the coarse and per-group fine models are trained on it. Abstain calibration holds out **10% of cell types entirely** as out-of-distribution (OOD — types the model was never trained on, standing in for the novel populations a real query contains), plus a 20% within-type test split, and sweeps the confidence threshold to trade coverage against accuracy on kept cells — the table each bundled reference reports.

Provenance and verification. Each reference carries a build record: census release, corpus sizes, hierarchy route, organism and calibration table. Because a rebuild can train cleanly and annotate *worse*, the previous reference is backed up and the new one is scored on a held-out atlas before it is kept. For `broad_mouse_v1` the test set is carved from the census itself: two datasets are excluded from the reference sample and re-pulled as the query, so "held out" means held-out *datasets* and not merely held-out cells.

2.7 Distilling a pretrained annotator

A broad reference can also be built without labels and without a GPU, by taking both the label vocabulary and the hierarchy from an existing pretrained annotator. Keras/TensorFlow and JAX cannot share a process, so this runs in two stages.

Teacher pass. Pan-human Azimuth labels a corpus of raw counts through its low-level API in batches of 8,192, which loads the weights once rather than once per call. All eight levels are retained. Where the package cannot reconcile a refined level with its coarser ones it blanks that column; those cells still carry a deepest-level call, so we fall back to it rather than discard them — dropping them would distill only the cells the teacher found easy. Predictions are mapped to CL ids through the package's own crosswalk.

Student pass. The student is a hierarchical actinn-jax reference whose **classes** are the teacher's refined fine labels (including its trained `Unassigned` quality-control class) and whose **hierarchy** is the teacher's own broad level — so the coarse→fine structure is inherited rather than rediscovered. 4,000-gene HVG panel; classes with fewer than 8 cells dropped, since a class that cannot be split into train and test makes fidelity depend on which side its cells landed.

Corpus. Three local atlases (lung, liver, blood+gut) capped at 600 cells per label, plus a census-wide pull ($\leq$60 per type). The atlases' own labels are used **only for scoring**, never for training.

Evaluation. Two arms answer different questions. *In-corpus* holds out a stratified 25% of cells and measures **fidelity** — how often the student reproduces the teacher, exactly and ontology-equivalently. *Held-out atlas* withholds an entire atlas and measures whether the student generalizes to data the corpus never covered. Both arms also score student and teacher against the atlases' own labels, which separates distillation loss from teacher error. Once the census pull is in the corpus, a withheld *atlas* is no longer a withheld *tissue*.

2.8 Refinement, abstention, and novelty

Four mechanisms operate on a trained reference at inference time, all label-free.

- **Confidence abstention.** Cells whose final probability is below a threshold are relabeled "unknown" rather than forced to the nearest class. Calibrated per reference (§2.6).
- **Refinement to the query.** The reference's class set is masked to those classes the query's own predictions actually support, and the softmax renormalized — no ground truth, no

retraining.

- **Refinement to a tissue.** Classes are masked to those the census records in the given tissue, while **pan-tissue** types (immune, endothelial, stromal) stay available everywhere, so liver-resident T cells are unaffected. The per-class tissue map is baked into the bundled reference; the tissue is given explicitly or read from the query's own metadata, with common synonyms (PBMC→blood, hepatic→liver) recognized. A tissue outside the reference's vocabulary imposes no filter rather than an empty one.
- **Cluster-level novelty screen.** A cell type absent from the reference appears not as scattered uncertain cells but as a *coherent group the reference cannot confidently explain.* Clusters that are both large enough to be a real population and predominantly low-confidence are flagged, with marker genes reported for each. This is distinct from per-cell abstention.

Within-type state. The same machinery resolves structure below a cell-type label: a hepatocyte zonation model trained on portal→central zone labels, scored by within-one-zone agreement across donors and datasets.

2.9 Scaling protocol

A benchmark run at one reference size measures that size, not the method. Since a reference can be anything from a few thousand cells to a whole atlas, we subsample the reference and hold the query fixed, so accuracy, fit time, predict time and peak memory become functions of reference size alone.

Each sweep answers a different question. **Cost against size and cardinality** (§3.3, Figure 2) covers six reference sizes to 24k cells and separately varies the number of cell types, and asks whether inference cost grows with either — the property the multi-stage workflow depends on. **Accuracy and memory to atlas scale** covers five sizes to 49k cells on lung and four to 47k on the HLiCA liver atlas (§3.3, Figure 3), and asks whether the ranking at laptop size survives at atlas size. A third sweep runs the other way, holding the reference fixed and growing the *query* to a whole 525k-cell atlas (§3.3, Figure 4), which is the axis a reused reference is actually pointed along. Sizes are set by capping cells per type, which is why they are not round numbers. The lung sweep's top point is the whole atlas once the query is held out; the liver atlas is far larger (525k cells), so its sweep stops at a matching reference size rather than at its own ceiling, which is what lets the two be read against each other.

2.10 External validation protocol

Open Problems `label_projection` [Open Problems 2024] supplies the datasets, metrics and ranking — none chosen by us. actinn-jax is packaged as a viash 0.9.7 component declaring the normalization it expects, and run through the project's own Nextflow workflow. We additionally built components for four baselines (linear-anova-pca, scTOP, SVM, CellTypist) so the same-hardware comparison covers more than the leaderboard's own set.

Where each number is measured. The paper reports two environments. The in-house panel (§3.1–§3.5) runs on the Apple Silicon laptop described in §2.5. The Open Problems results (§3.7) run on AWS `r7i.8xlarge` instances (32 vCPU, 247 GB) through OP's own Nextflow pipeline, two tasks at a time.

Accuracy in Table 9 comes from a single AWS run covering all eleven methods, so every score is one harness, one machine, one invocation.

Cost is reported as a ratio to actinn-jax rather than in seconds, because wall-clock on a shared box depends on what else is running: measured `%cpu` spans 49% to 2338% across these methods, and actinn-jax itself averaged 165 s/dataset when scheduled alongside OP's seven and 87 s/dataset alongside our four. Ratios taken within one run are stable where absolute times are not, and actinn-jax was included in both runs to join them. Peak memory does not move with contention and is reported as measured.

2.11 Ablations

Three cheap interventions characterize *when* a gene-space MLP gains or loses against heavier methods, all selectable without test labels:

- **Input standardization** — z-score each selected gene by the reference's frozen mean/std and apply to the query (a scArches-style domain alignment). Opt-in, because it shifts the probability calibration the abstain thresholds are tuned against.
- **Gene budget** — Open Problems feeds every method 1,000 HVGs; we sweep wider panels and select per dataset by **held-out reference cross-validation**, which is label-free with respect to the test set.
- **Protein-embedding featurization** (negative control). A CPU-only featurization in the style of **Universal Cell Embeddings (UCE)** [Rosen 2026]: an expression-weighted mean of ESM2 [Lin 2023] protein-language-model gene embeddings. It tests whether a foundation model's value survives a cheap pooling shortcut.

3 Results

3.1 Accuracy and cost

Accuracy is the mean over the **five shared-vocabulary datasets** (lung_cross excluded — its exact-match accuracy is a vocabulary artifact, see the † note below; means over all six are ~0.08 lower for every method, and reorder only scArches and actinn-jax, by 0.0004); macro-F1 is the mean over all six non-brain datasets, ontology concordance the mean over the four of those that carry Cell Ontology ids; cost is the mean per query (Table 3). The two brain splits are excluded from these aggregates and reported per dataset in Table 5, because the columns below come from one measurement pass on an idle machine and brain was run afterwards. Adding brain at subclass level raises every method by 0.025–0.043 and reorders nothing. Adding both brain splits gives accuracies over eight of linear-anova-pca 0.857, scANVI 0.855, scArches 0.853, CellTypist 0.848, **actinn-jax 0.840**, SVM 0.837, ProtoCloud 0.817, SingleR 0.811, kNN 0.792, scTOP 0.775, scmap-cluster 0.706 — which moves actinn-jax from fourth to fifth, behind CellTypist:

method	acc (5)	macro-F1 (6)	ontology (4)	fit (s)	**predict (s)**	peak mem (MB)
linear-anova-pca	**0.839**	0.699	0.808	**3.4**	**0.33**	4386
scArches	0.833	**0.701**	0.804	48.7	17.2	1773
scANVI	0.832	0.698	0.808	0.0*	66.7	2087
actinn-jax	0.831	0.683	**0.811**	24.2	**0.54**	2391
CellTypist	0.823	0.690	0.805	54.4	**0.61**	1569
SVM	0.808	0.675	0.796	55.2	**0.07**	1419
ProtoCloud	0.790	0.649	0.778	221.5	2.07	1907

method	acc (5)	macro-F1 (6)	ontology (4)	fit (s)	**predict (s)**	peak mem (MB)
SingleR	0.770	0.652	0.750	0.3	48.2	3612
kNN	0.770	0.623	0.768	0.4	**0.19**	1483
scTOP	0.739	0.619	0.703	1.1	1.21	1926
scmap-cluster	0.646	0.550	0.771	0.3	9.30	8609

Table 3. Accuracy and cost together, ordered by accuracy. Accuracy is the mean over the five shared-vocabulary datasets, macro-F1 is the mean over all six non-brain datasets, ontology concordance the mean over the four of those carrying Cell Ontology ids, and cost is the mean per query. The two brain splits are reported per dataset in Table 5 instead (see above). Bold marks the best value in a column.

*In Table 3, scANVI does most of its work in one train+predict pass, attributed to predict.

The top four methods span **0.008 in accuracy, ~205× in predict time** (0.33 s to 67 s) and 2.5× in memory. That accuracy span is not a ranking and should not be read as one: the stochastic methods move further than 0.008 between identical reruns on their own — scANVI by up to 0.056 across its three repeats — so which of the top four places second and which places third is decided by seeds, not by method. The cost column is what separates them, and scANVI is **123× slower than actinn-jax** at accuracy that is tied within that noise. The linear pipeline is both the most accurate and the fastest to fit, and pays for it in memory — 4386 vs 2391 MB — because ANOVA/PCA densify a cells × genes matrix while actinn-jax stays sparse; that ~1.8× ratio widens to ~2× at atlas scale (6.1 vs 13.2 GB at 49k cells, §4). The two profiles suit different jobs: one-shot labeling, versus a reference **trained once and reused**, where a cached model pays fit once while the linear pipeline refits scaler/PCA/classifier for every query. ProtoCloud's 222 s CPU fit buys nothing at this reference size — and buys the top accuracy at atlas scale (§3.3).

Pretrained annotators, scored ontology-only. The two methods with fixed label vocabularies (§2.2, §2.4) cannot appear in Table 3, since they do not predict the dataset's label strings. They are scored by ontology concordance on the three datasets where reference and query both carry CL ids and share a vocabulary, with actinn-jax on the same splits for reference:

	lung_intra	liver_intra	liver_cross	predict (s)
actinn-jax (reference-trained)	**0.917**	**0.846**	**0.731**	0.27–0.37
Pan-human Azimuth (pretrained)	0.700	0.521	0.408	2.2–3.4
scPRINT (zero-shot)	0.201	—	—	62.3

Table 4. Pretrained annotators, scored by ontology concordance only, on the three datasets where reference and query both carry Cell Ontology ids and share a vocabulary — with reference-trained actinn-jax on the same splits for scale. Not a like-for-like comparison; see the text below.

This is not a like-for-like comparison and should not be read as one. actinn-jax is trained on a reference drawn from the same data and scored in its own vocabulary; Pan-human Azimuth has never seen these datasets and answers in a different one. A reference-trained model *should* win. What the block does establish is the distance between a curated supervised annotator and a zero-shot foundation head — 0.700 against 0.201 on lung — and it sets up §3.4, where Azimuth serves as a broad pass in its own right and as a distillation teacher. Peak memory is indistinguishable between actinn-jax and Azimuth (1.65–2.1 GB on every dataset).

Those four are the tuned **linear pipeline (0.839)**, scANVI (0.833), scArches (0.832) and actinn-jax (0.831) — led by the linear pipeline rather than by a deep model. CellTypist (0.823) sits just outside them. actinn-jax has the best ontology-aware concordance (0.811), marginally ahead of scANVI's 0.809 — a gap well inside repeat noise, so read it as a tie rather than a lead. ProtoCloud (0.790) and scTOP (0.739) sit below that cluster on these subsampled references: both need conditions these splits do not provide — ProtoCloud is data-hungry and becomes the strongest method once given a real atlas (§3.3), while scTOP is built for small, low-cardinality problems. Per dataset (accuracy):

dataset	actinn-jax	best method (acc)
lung_intra (46 types)	0.894	ProtoCloud 0.932
lung_cross (cross-dataset)†	0.358 exact / 0.752 ontology	ProtoCloud 0.374 / **0.791 ontology**
liver_intra (36 types)	0.802	linear-anova-pca 0.804
liver_cross (cross-study)	**0.686 exact / 0.731 ontology**	actinn-jax
blood_gut (86 types)	0.860	linear-anova-pca 0.902
pbmc (8 types)	0.913	scArches 0.940
brain, subclass (24 types)	0.986	ProtoCloud 0.991
brain, cluster (151 types)‡	0.741	SingleR 0.845

Table 5. Per-dataset accuracy: actinn-jax against whichever method leads that dataset. † on lung_cross the exact-match score is a vocabulary artifact, so ontology concordance is the meaningful column. ‡ the two brain rows are the same nuclei scored at two resolutions of the same taxonomy; see below.

No single method leads everywhere: ProtoCloud takes three splits, the linear pipeline two, and actinn-jax, scArches and SingleR one apiece. actinn-jax leads the cross-study liver split — the hardest generalization regime here, and the one closest to real reference mapping — and is second on within-dataset liver, while trailing on lung, on the 86-type blood+gut set (−4.2 pt to the linear pipeline), and on small-n pbmc. Outside brain, the spread across the leading methods on any one dataset is 1–4 points, and scmap-cluster is the weakest on accuracy and macro-F1 throughout (0.23 macro-F1 on liver_cross), though not on ontology concordance.

The two brain splits are the same nuclei at two resolutions. At Allen's 24 subclasses ten of the eleven methods fall inside 0.017 of each other (0.974–0.991, with scmap-cluster alone below at 0.901) and no method moves more than 0.003 between repeats, so that split does not separate them. At the 151 clusters the ranking **inverts**. SingleR leads at 0.845, having placed eighth of eleven overall; scmap-cluster, last everywhere else at 0.646, reaches 0.810; and actinn-jax falls to 0.741, tenth. The two correlation methods gain where the trained classifiers lose.

We checked three explanations for that. Every method ran at the same defaults as on the other six splits. Raising actinn-jax's 50 training epochs to 150 and then 400 moves it 0.741 → 0.746 → 0.747, so eight times the training buys 0.006. And the reference is not unusually thin: this split gives 72 reference nuclei per class against 89 on blood+gut, where actinn-jax scores 0.860. What remains is cardinality, and the provenance of the labels: **cluster-level labels are the output of clustering this same expression space**, which favors methods that classify by distance to a class centroid. Methods also disagree about *which* clusters are hard (Supplementary Figure S5): median best-minus-worst recall 0.32 per class, mean pairwise Spearman 0.61, concentrated in the graded `L2/3 IT`, `Sst` and `Vip` families rather than in the rare classes.

† lung_cross exact-match accuracy (~0.35 for every method) is a label-vocabulary artifact, not a transfer failure. HLCA (reference) and Krasnow (query) were annotated independently and share only **20 of 46 cell-type names**; 26 of Krasnow's types are absent from HLCA's vocabulary, so no classifier can emit the exact string. The mismatch is granularity: HLCA's finer taxonomy has `alveolar / elicited / lung macrophage` but **no generic `macrophage`** (CL:0000235), while Krasnow labels many cells generic `macrophage` — so an HLCA-trained model predicts `lung macrophage` (CL:1001603) for them, exact-wrong but ontology-correct (*lung macrophage is-a macrophage*); likewise Krasnow's generic `endothelial cell` (CL:0000115) has no HLCA counterpart. **Ontology-aware concordance — which credits a same-lineage call — is ~0.75 for the trained methods (0.67–0.79 across the full panel, highest for ProtoCloud)**, so cross-dataset transfer actually works; exact-match just conflates classifier error with vocabulary/granularity mismatch. This is precisely why we report ontology concordance, and it is a general benchmarking pitfall: **cross-dataset exact-match accuracy between independently-annotated atlases is not a meaningful accuracy signal.** The other five datasets use a single shared label vocabulary for reference and query (a split of one atlas, or HLiCA's harmonized annotations), so their exact scores are unaffected.

3.2 The accuracy × speed frontier

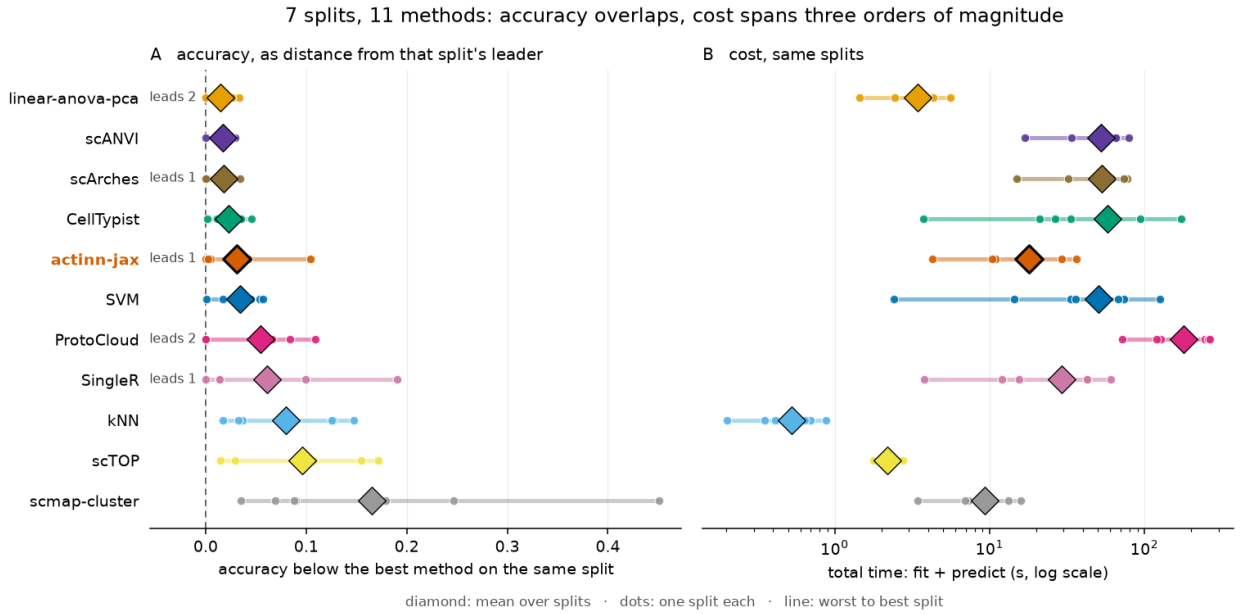

Figure 1. Eleven methods over seven splits (lung_cross excluded, see the † note above). *A:*

accuracy as the **gap to whichever method leads that split**. Raw accuracy is dominated by how hard a split is — 0.94 on pbmc against 0.36 on cross-dataset lung — so a range drawn over raw accuracy would measure the datasets rather than the methods; zero is that split's leader, and `leads N` counts the splits a method wins. *B:* fit + predict on the same splits, log scale. Diamonds are means over splits, small dots the individual splits, and the line runs from the method's worst split to its best. Ranges over the three repeats *within* a split are too small to draw here: only scANVI (0.056) and scArches (0.020) move more than 0.009, and five methods are exactly deterministic. Per-split detail, unnormalized and with those repeat ranges, is Supplementary Figure S11.

Six methods sit within **0.035** of the per-split leader on average while spanning **3.4 s to 57 s**, and the full panel spans **0.5 s to 178 s**. The **tuned linear pipeline** is both closest to the leader (mean gap 0.014, leading two splits) and the cheapest of that group at 3.4 s; kNN and scTOP are faster still, at 0.5 s and 2.2 s, but give up 0.08 and 0.10, so they hold a different corner of the frontier rather than sitting inside it. The deep methods buy little on accuracy — scANVI and scArches place second and third by gap, 0.003 and 0.004 behind the linear pipeline, which is inside their own rerun noise — and pay 15× its total time for that. **actinn-jax** sits mid-panel: mean gap 0.031 at 17.9 s, leading the cross-study liver split. What separates it is not on this plot at all but on the scaling axes of §3.3 — predict time flat in reference size, and ~2× lower peak memory that holds to atlas scale.

Every method's spread across splits is larger than the distance between the leading methods: scmap-cluster spans 0.035 to 0.451, SingleR 0.000 to 0.191, actinn-jax 0.000 to 0.104. A mean over a panel of datasets, including the one in Table 3, summarizes that spread with a single number.

3.3 Scaling

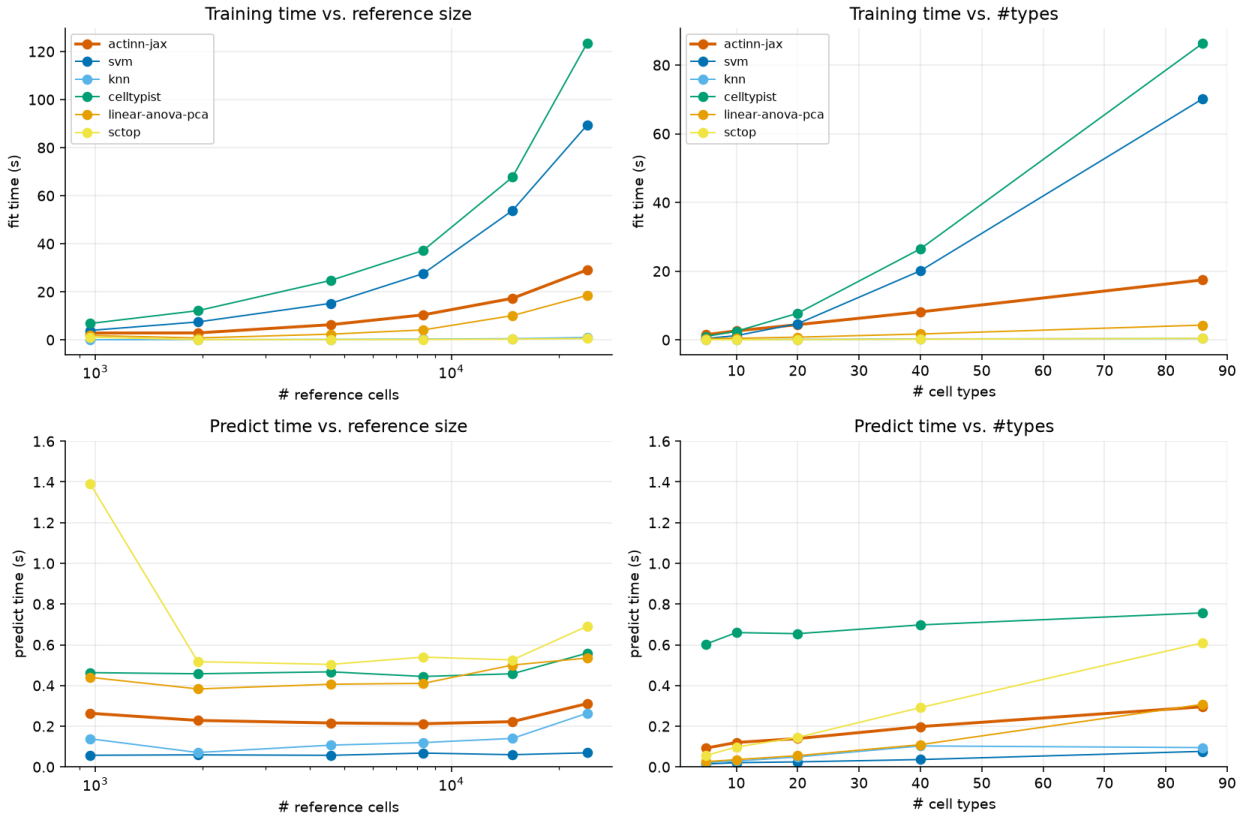

Figure 2. Fit and predict time against reference size and label cardinality, for the six CPU methods. Fit grows with both axes for every trained method, steeply for CellTypist and the SVM. Predict is flat in reference size for all six, so that flatness is a property of cached inference generally rather than of any one method; the axis that separates them is cardinality, where actinn-jax rises 3.2× from 5 to 86 types against the tuned linear pipeline's 11.9×, the two meeting at ~0.30 s. SVM and kNN predict faster than either throughout. scTOP's smallest-reference point carries one-time import cost and is not a scaling effect.

Fit time grows with reference size and with #cell types for all trained methods — actinn-jax's fit goes 3 s → 17 s → 29 s as the reference grows 965 → 14.8k → 24k cells, below CellTypist (7 s → 68 s → 123 s) and SVM (4 s → 54 s → 89 s) at every size in the sweep, and above the tuned linear pipeline (2 s → 10 s → 19 s); kNN does no work at fit time beyond storing the reference, so all of its cost lands on predict, where it is also the least accurate method here.

On the other axis, **predict stays sub-second for every method across the whole sweep**, and the first thing to say is that flatness in reference size does not distinguish anything: all six move by 1.2–1.9× while the reference grows twenty-five-fold, because a cached model's inference does not depend on how many cells trained it. Cardinality is where they part. actinn-jax rises 3.2× from 5 to 86 types (0.09 → 0.30 s); the tuned linear pipeline starts three and a half times cheaper and rises 11.9× (0.03 → 0.31 s), so the two arrive at the same place; scTOP rises 11.0×. Neither of the two cheapest is ours — SVM (0.015–0.076 s) and kNN (0.021–0.263 s) predict faster than actinn-jax everywhere, and pay for it in accuracy (Table 3).

What the cached reference buys is therefore not a uniquely flat predict but the right *ratio*: a sub-second call recurring against a fit of 19–123 s that is paid once. That is the regime that matters when one reference serves many queries, and it is what makes chaining several annotation stages practical; it would be as true of the linear pipeline if it cached its scaler, PCA and classifier rather than refitting them per query. (This sweep's own memory column is process-cumulative and not a clean per-size measurement; the atlas sweep below and the matrix of §3.1 both run each method in its own process, and are what the memory numbers come from.)

At atlas scale the ordering changes. The sweep above stops at 24k cells. Carrying four methods to full atlas size — 49k reference cells on lung, 47k on the HLiCA liver atlas — reverses the accuracy ranking (Figure 3A, B). ProtoCloud goes from the weakest method at 3k cells (0.722 lung, 0.581 liver) to the strongest at full scale (**0.976** and **0.905**), clear of the linear pipeline (0.939, 0.863) and actinn-jax (0.936, 0.824), at 19× actinn-jax's CPU fit cost. scTOP gains little from scale on lung (0.819 → 0.846, barely outside its interval) and loses ground on liver (0.657 → 0.583). Peak memory grows for every method, but the spread stays **bounded at ~2**× rather than widening (Figure 3C): the linear pipeline is heaviest throughout — 13.2 GB against actinn-jax's 6.1 GB at 49k lung cells, a 2.15× ratio — and scTOP crosses over from lightest to heaviest as its rank processing densifies (9.3 vs 6.5 GB at 47k liver cells). Conclusions drawn from subsampled references do not transfer to atlas scale, in either direction.

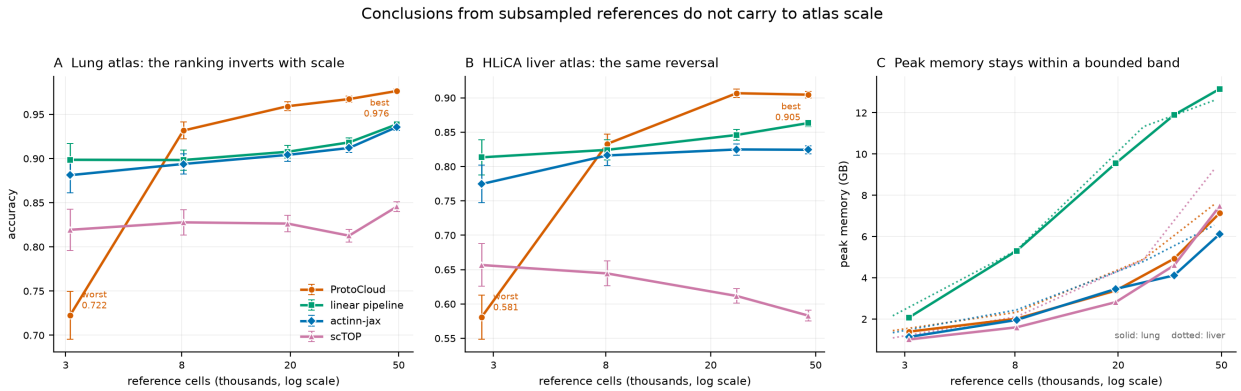

Figure 3. Accuracy and peak memory against reference size, four methods carried to full atlas scale on two datasets. *A:* lung, 3k → 49k reference cells. *B:* the HLiCA liver atlas, 2.7k → 47k, an independent replication of the same reversal. *C:* peak memory over both sweeps. Error bars on *A* and *B* are 95% binomial intervals on the query, which grows with the reference (1,035 to 16,398 cells on lung), so they narrow from left to right; each point is a single run, so they cover sampling of the query and not run-to-run variation. Peak memory is not a proportion and carries none. Fit time is not plotted: the 47k liver point was measured under CPU contention, so a cost axis there would report scheduling rather than scaling.

Annotating an atlas costs less than building one. The sweeps above grow the reference and hold the query fixed. The reverse case is the one a user meets most often – a reference built once, then pointed at everything – and it is where a flat inference profile should pay. Holding the reference at 17,753 cells drawn from seven HLiCA studies and growing the query from 50k cells to the entire 524,699-cell atlas, actinn-jax annotates the whole atlas in **41 s** (Figure 4A). Accuracy on the withheld eighth study is 0.720 and does not move across the range, as it should not: the subsets are nested.

The tuned linear pipeline stays three to four times slower throughout, annotating the same atlas in 126 s, but the gap narrows as the query grows and the reason is ours rather than theirs: actinn-jax gives up 31% of its rate across the range, 18,400 → 12,800 cells/s, while the linear pipeline holds essentially flat at ~4,200, losing 6% (Figure 4B). The advantage is 4.2× at 50k cells and 3.1× at the full atlas. It is the cheaper method whose per-cell cost drifts upward here, which is the opposite of what a flat-inference argument predicts, so the claim this axis supports is a three- to fourfold constant factor rather than flatness. The flat-inference result in §3.1 is a *reference*-axis result and should not be read onto this one.

This inverts the Table 3 ordering, and the two measurements do not overlap. Every query in this sweep holds 50,000 cells or more; every query in Table 3 holds 13,550 or fewer, so no query size is measured in both, and the crossover falls in the untested gap between them.

Within Table 3's range the ordering is mixed rather than uniform. The linear pipeline predicts faster on six of the eight splits, by 1.3× to 3.1×, and actinn-jax is faster on liver_cross (0.217 s against 0.267 s for 3,396 cells) and level on the brain cluster split (0.363 against 0.372 for 3,618). Query size does not order those outcomes — actinn-jax wins at 3,396 cells and loses at 13,550 — because the two methods' per-cell costs depend on the gene panel each dataset selects. Table 3's 0.54 s against 0.33 s is a mean over that mixture, not a result that holds per dataset.

Part of the small-query gap is one-time cost. Calling `predict` repeatedly on one liver_intra

model and query settles from 0.24–0.30 s on the first call to 0.12–0.13 s by the third (two runs, `predict_overhead_probe.py`), so roughly 0.12–0.17 s is compilation and warm-up — about half the first call at 1,332 cells, and near a tenth of it at 13,550. The harness times the first call, which is what a one-shot annotation costs, but it means Table 3 charges actinn-jax a fixed cost that the linear pipeline does not pay.

At atlas scale a different mechanism dominates and the ordering stops being mixed. actinn-jax climbs from 4,100–15,700 cells/s on the Table 3 queries to 12,800–18,400 here, while the linear pipeline falls from 9,700–13,000 to ~4,200, because its dense feature block leaves cache: 20,000 genes over 1,332 cells is 107 MB, and the same 20,000 genes over a 50,000-cell block is 4 GB and bounded by memory bandwidth. Which method is cheaper therefore depends on how many cells are annotated at once — which a benchmark reporting one query size cannot show.

Peak memory does not separate them at all. Both land at 26–28 GB on the full atlas (Figure 4C), because on this axis peak memory measures holding the query rather than running the method: at the largest size, prediction added nothing measurable on top of the resident query for either. A caller streaming from disk would pay less, and would pay it equally. The ~2× memory band reported above is likewise a *reference*-axis result and does not carry to this one; only the reference axis separates these methods on memory.

Two conditions of this measurement. Prediction in our linear adapter blocks the query into 50,000-cell chunks. Unblocked, it densifies 20,000 genes for the whole query at once, which needs roughly 126 GB at the full atlas and does not complete on a 51 GB machine — a property of the wrapper rather than of the recipe. Blocking leaves every prediction bit-identical, and is verified to. Second, these are laptop numbers, so each point is the fastest of three runs: a competing process can only add time, and stalls on this machine are large and sporadic rather than small and Gaussian – one repeat turned a 50 s fit into 975 s, and no mean over three runs would survive that. Peak memory takes the largest of three instead, since `ru_maxrss` counts resident pages and therefore reports *less* than a run needed whenever the OS evicts under pressure. Per-run values are released with the harness so the spread is inspectable rather than summarised away.

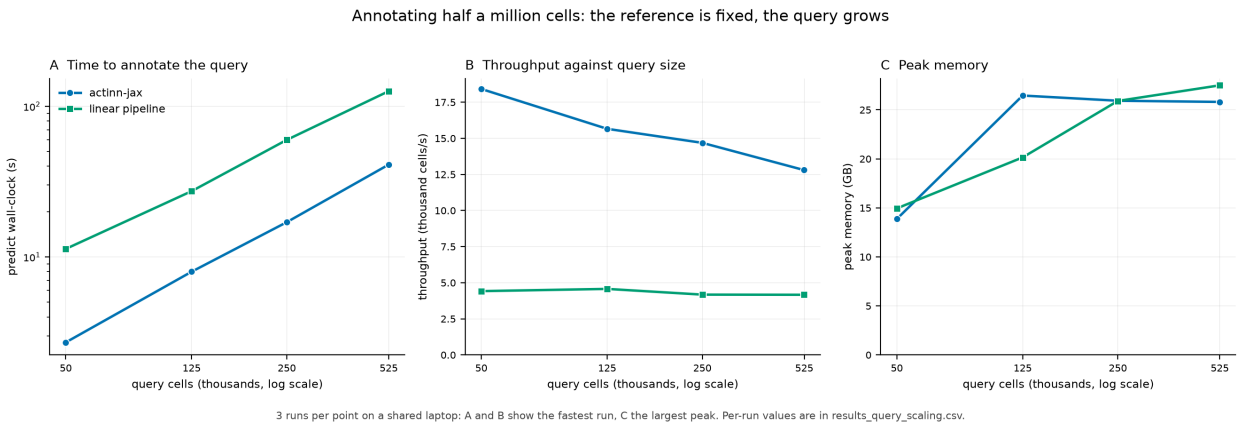

Figure 4. Cost against query size — 50,000 to 524,699 cells — with the reference fixed at 17,753 cells. Table 3's queries are all smaller than the leftmost point here (657 to 13,550 cells), so the two do not overlap. *A:* wall-clock to annotate the query. *B:* throughput, which declines 31% for actinn-jax and holds flat for the linear pipeline, narrowing the advantage from 4.2× to 3.1×. *C:* peak memory, which does not distinguish them – on this axis it measures holding the query, not running the method. Three runs per point on a shared laptop: *A* and *B* report the fastest run,

since contention can only add time, and C the largest peak, since resident-set size understates a run that the OS has partly evicted. Per-run values are in `results_query_scaling.csv`.

3.4 The broad→focused annotation workflow

What actinn-jax is built around is not a single classifier but a **two-pass workflow** that matches how annotation is actually done: get a broad call fast, then sharpen it where it matters. Because every stage is a cached `ReferenceModel` with sub-second, memory-bounded inference (§3.1, §3.3), the whole workflow runs on a laptop.

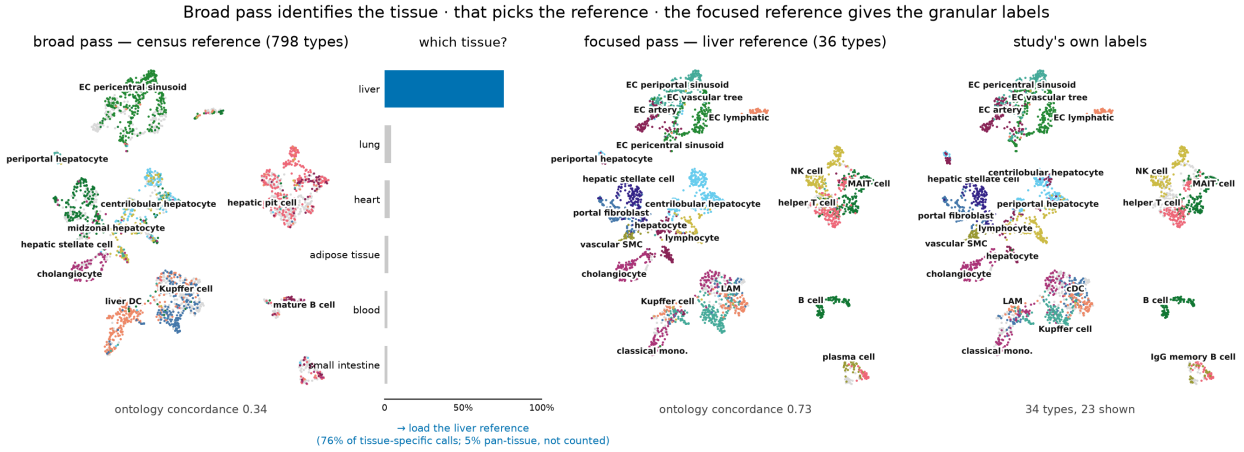

Figure 5. The workflow on a withheld HLiCA liver study (3,396 cells), same embedding throughout. *Left:* the census reference spreads 144 of its 798 labels over the query (concordance **0.34**) — it is not trying to name subtypes. *Center:* those calls resolved to tissue through the reference's per-class tissue map; **76%** of tissue-specific calls say liver against 4% for the next candidate, which is the decision that selects the next reference. *Right of center:* the 36-type liver reference re-annotates the same cells at **0.73**, now tracking the clusters. *Right:* the study's own labels on the shared palette. Both models are leakage-free for this query: the focused reference is trained only on the six non-withheld studies. Abstention is off in all panels, so the numbers measure labeling rather than coverage; §3.5 covers abstain separately.

The broad pass — census-scale. A ~800-type human reference built from the CELLxGENE census gives any query a first-pass annotation across the whole body, with a **calibrated abstain threshold** so out-of-reference cells are flagged rather than force-labeled (§3.5). This is the orientation pass — broad coverage, deliberately not fine-grained.

The broad pass can be built from a model instead of from labels. The census route above needs a foundation model on a GPU to discover its hierarchy. A pretrained pan-human annotator already has one — Pan-human Azimuth publishes an 8-level typology with every node mapped to a Cell Ontology term — so labeling a corpus with it and training actinn-jax on those labels transfers both the vocabulary and the structure. That build needs **no GPU and no labeled input**, only raw human counts: under ten minutes of CPU on 85k cells drawn from three atlases plus a census-wide sample. On 3,396 withheld cross-study liver cells:

broad-pass model	classes	ontology	cells/s
census-built (`broad_human_v1`)	798	0.338	2,962

broad-pass model	classes	ontology	cells/s
Pan-human Azimuth	382	**0.408**	1,076–1,563
distilled (`panhuman_distill_v1`)	324	0.406	**8,937–10,021**

Table 6. Three broad-pass entry points on 3,396 withheld cross-study liver cells: the census-built reference, the Pan-human Azimuth teacher, and the model distilled from that teacher. All three are scored on identical cells through one script (below), with every call mapped to a Cell Ontology id at prediction time: scoring a cached prediction dump instead leaves 9.5% of Azimuth's calls unmapped, and an unmapped call scores as wrong, which costs it 0.028 concordance it did not lose to the student.

This makes the entry point as accurate as its teacher and ~6–9× faster, and more accurate than building it from the census directly, and it answers in a harmonized, ontology-mapped vocabulary rather than raw census strings. Withholding a whole atlas from the corpus, the distilled model tracks the one it was distilled from to within **1.5 points** on lung (0.695 vs 0.710) and **3.0** on liver (0.481 vs 0.511). Distillation itself therefore costs very little: on data neither model was built for, the student stays within a few points of its teacher. What holds the student back is which tissues and studies its training corpus happened to contain — widening the corpus should move these numbers, changing the distillation procedure should not. Two caveats keep this modest. Student and teacher are level on this query (0.406 against 0.408), and both actinn-jax models draw on a census sample that may include these studies while Pan-human Azimuth does not, so read the distilled model as *comparable to* the one it distills rather than better. And it does **not** inherit that model's trained abstention: its confidence separates right from wrong poorly (keeping the 90.5% of cells it calls with probability at least 0.5 moves concordance only 0.406 → 0.427), so the calibrated broad-pass abstain of §3.5 belongs to the census-built reference until this one is recalibrated.

The focused pass — tissue-specific. For the tissue the broad pass identifies, a small focused reference re-annotates at full resolution. Holding out a whole HLiCA study (56,545 cells, a different research center's protocol), the broad census model scores exact-match **0.23** / ontology **0.58**, while a focused **38-type HLiCA liver** reference on the *same cells* reaches **0.72 / 0.86**. Refinement is where fine-grained accuracy comes from; the broad model's job is to route to it, not to be right about subtypes itself.

The broad pass hands the query to the focused pass; the two do not combine. The natural assumption is that a better broad call should also make the focused call better. It does not. On the leakage-free cross-study liver split, substituting the stronger **Pan-human Azimuth** for the broad pass lifts it (ontology 0.408 vs 0.338 for our own broad model) but changes nothing downstream. Using that broad call to *narrow* the focused pass's classes — the zero-retrain masking actinn-jax provides — makes the result **worse**, 0.731 → 0.708: the broad call matches the true lineage on 85.8% of cells, and the 14% it misses cost more than the 86% it gets right can gain, since a wrong mask discards the correct class outright. A **perfect** broad call would add only **+2.8 points** (0.759). Once the focused pass covers the tissue, there is almost no accuracy left for a broad model to contribute; its value is choosing which focused reference to load, and catching cells that fall outside every focused reference's scope.

Can a broad reference be built for an organism with no pretrained annotator? For mouse, neither human route carries over intact, but for different reasons. Distillation needs a teacher, and Pan-human Azimuth is human-only — there is no mouse model to distill. The census

route does apply to mouse, but its one expensive step does not travel cheaply: the coarse groups are found by Ward-clustering scPRINT embeddings of per-type expression centroids, which wants a GPU, and scPRINT's mouse support is untested here.

That clustering step is the only one that has to be replaced (Figure 6A). The census already labels every cell with a Cell Ontology term, and CL encodes the relation the embedding clustering is recovering — which cell types are kinds of the same thing. Describing each type by the CL terms it descends from and clustering *those*, with the same Ward linkage, gives a hierarchy that is free, deterministic and species-independent.

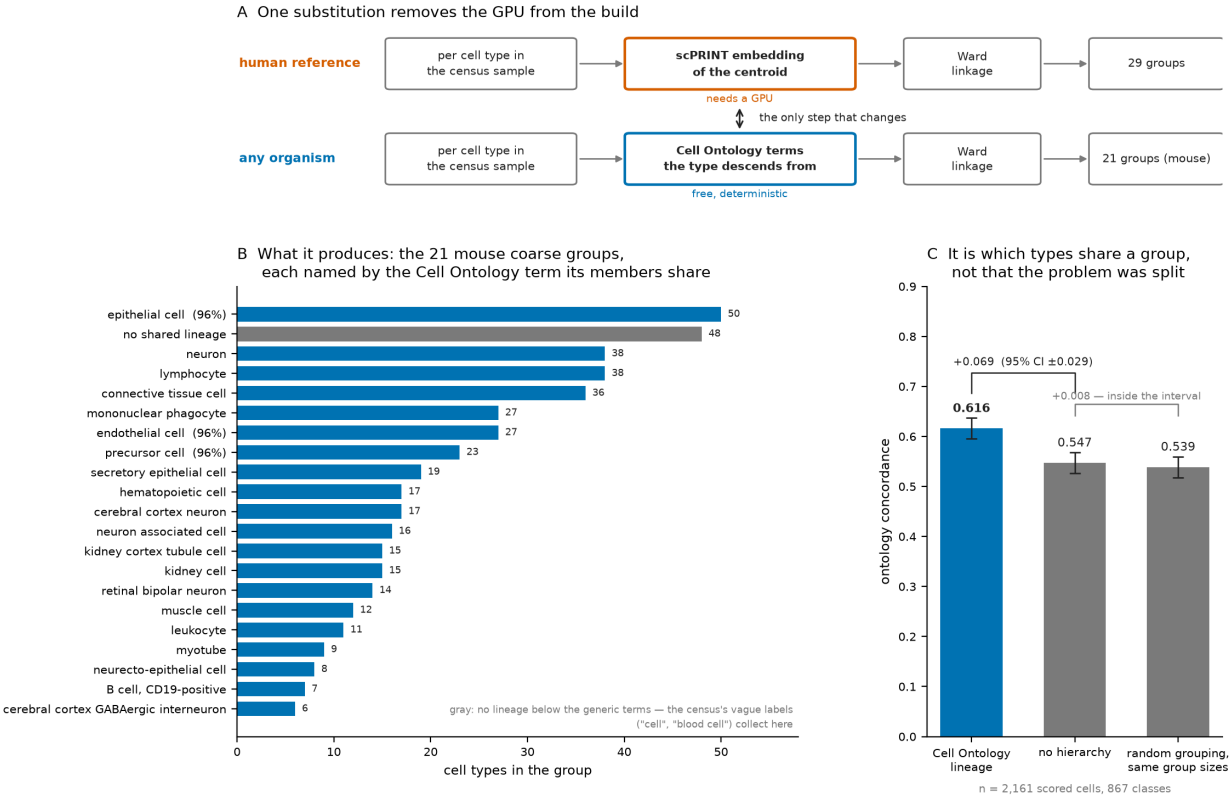

Figure 6. Replacing the one step of the reference build that needs a GPU. *A:* both routes to the coarse groups; only the middle step differs. *B:* the 21 coarse groups of `broad_mouse_v1`, each named by the Cell Ontology term its members share, with a percentage where that term covers most but not all of the group; the gray bar is the single group whose members share no term below the generic ones. *C:* the ontology grouping against no hierarchy and against a random grouping with the same group sizes, scored by ontology concordance on the held-out human lung atlas. Bars start at zero and error bars are 95% binomial intervals on the 2,161 scored cells; the gain over no hierarchy clears that interval and the gap between the two controls does not.

The groups it produces are lineages. Twenty of the 21 mouse groups resolve to one CL lineage each — lymphocytes, neurons, epithelium, endothelium, mononuclear phagocytes, kidney tubule — and the twenty-first is a catch-all of 48 types whose census labels (`cell`, `blood cell`) are too vague to place anywhere (Figure 6B).

The grouping is doing the work, not the splitting. Any partition gives each fine classifier a smaller problem, so the substitution needs a control. Built from one human corpus (51,346 cells

/ 867 types) and scored on the held-out lung atlas, the ontology grouping reaches **0.616** against **0.547** for no hierarchy at all — a gain of **0.069**, or 13% relative, which on 2,161 scored cells sits outside the 95% interval of ±0.029. A random grouping with the same group sizes scores **0.539**, which is 0.008 from the no-hierarchy control and inside that interval (Figure 6C). What matters is *which* types share a group. Each group has its own fine classifier and the coarse call decides which one ever sees the cell, so a group pays off only if the coarse classifier can recognize the group *and* the right answer is inside it. Lineage groups manage both; same-sized random groups manage neither.

On that basis, `broad_mouse_v1`: 27,026 cells → **453 cell types across 85 tissues**, 305 CL terms collapsed to 21 coarse groups, **17 seconds of CPU** to train, 38 MB. Two mouse datasets were excluded from the reference entirely and used as the test set — 12,646 cells, 137 truth types, 41 tissues, none of it seen in training:

	ontology	coverage	cells/s
all cells	0.638	100%	9,712
confidence ≥ 0.5	**0.718**	71%	

Table 7. `broad_mouse_v1` on 12,646 held-out mouse cells (137 truth types, 41 tissues, none of it in the reference), with and without abstain.

Its abstain calibration is also better behaved than the human census model's: at the same `p ≥ 0.5` threshold it still answers for **71%** of cells (Table 7), against **38%** for `broad_human_v1` on its own held-out atlas, because mouse census carries fewer near-duplicate subtypes than human's ~800-way vocabulary. Coverage at a fixed threshold is the part that compares across the two — the accuracies come from different queries and should not be read head-to-head. Two limits apply. The ablation above was run on **human** and applied to mouse; CL is species-neutral by construction, but nothing here shows it groups mouse types as well. And mouse census is shallow in *datasets* — 51 in total, one embryo atlas holding 11.4M of 18.4M cells — so tissue breadth is good while lab and protocol breadth is far below human's 487 datasets.

Supporting mechanisms. (i) The **embedding route to the same hierarchy**, evaluated separately from the ontology route above: a foundation model discovers the coarse→fine structure *offline* and the CPU model uses it at inference. It beats a flat classifier on three of four datasets, matches a hand-built expert hierarchy, and beats a random-grouping control, while never calling the foundation model at prediction time. (ii) **Within-cell-type resolution**: the same machinery resolves hepatocyte zonation (portal→central) at 0.99 within-one-zone across held-out donors, and 0.88–0.92 transferring between datasets — a third stage below the cell-type label. Together these make actinn-jax a *pipeline* (broad → tissue → subtype/state), not just a classifier, at classical-method speed throughout.

Running all three: where independent references agree. The three entry points are interchangeable, so we ran all of them over the same query. They answer in three different vocabularies (798, 382 and 324 classes), so agreement is defined in the Cell Ontology where all three map: two calls agree when they are the same term or one is an ancestor of the other, the relation the concordance metric already uses. Partitioning by how many of the three mutually agree, on the withheld cross-study liver query and on the 65,662-cell Krasnow lung atlas:

tissue	tier	coverage	census	distilled	Azimuth
liver	all three agree	23%	0.690	0.778	0.785
liver	two agree	55%	0.241	0.303	0.311
liver	none agree	22%	0.212	0.276	0.257
liver	*whole query*	100%	*0.338*	*0.406*	*0.408*
lung	all three agree	48%	0.934	0.917	0.917
lung	two agree	50%	0.156	0.768	0.770
lung	none agree	3%	0.059	0.436	0.425
lung	*whole query*	100%	*0.524*	*0.830*	*0.831*
brain	all three agree	94%	0.839	0.997	0.997
brain	two agree	6%	0.107	0.829	0.869
brain	none agree	1%	0.117	0.106	0.568
brain	*whole query*	100%	*0.792*	*0.981*	*0.987*

Table 8. Ontology concordance within each agreement tier, three broad references on identical cells, in three tissues: withheld cross-study liver (3,396 cells, 34 truth types), the Krasnow lung atlas (65,662, 46) and the Allen human middle temporal gyrus (156,285, 18).

One thing replicates in all three tissues: cells the references disagree about are annotated far less reliably than cells they concur on, for every reference. The census model spans 0.690 to 0.212 across the liver partition, 0.934 to 0.059 across lung and 0.839 to 0.117 across brain, with the same direction for the other two. That all three improve *together* is what makes the partition useful — agreement selects cells that are unambiguous rather than cells one model happens to get right — and unlike accuracy it can be computed on a query whose answers are unknown, for the price of three sub-second calls.

What does not replicate is how much of a query the agreeing set covers: 23% on liver, 48% on lung, 94% on brain. Brain explains the spread rather than extending it. That query's Cell Ontology annotation uses **18 terms for a region whose own taxonomy, `CCN201908210`, defines 154 cell sets** — 151 of them present in this data, and the label set of the fine brain split in §3.1 — while 55% of its cells fall in one class, `L2/3-6 intratelencephalic projecting glutamatergic neuron`. Concordances near 0.98 and agreement at 94% are what a coarse truth vocabulary produces, not what an easy tissue produces. The partition reports the resolution of the annotation it is scored against, which is a property of the query rather than of the method: it reports how much of a query is unambiguous at the resolution that query is annotated to.

The consensus *label* is not the prize in any of the three, and brain shows why the rule itself is weak. Taking the most specific call the agreeing references support scores 0.365 on liver against the best single reference's 0.408, 0.828 on lung against 0.831, and 0.837 on brain against 0.987. That last gap is large because "most specific agreeing call" lets one reference's confident, lineage-compatible but wrong specificity override two correct coarser calls: on the 94% of brain cells where all three agree, the two strong references score 0.997 and the consensus built from them scores 0.837. A better rule may exist; what the experiment establishes is that the *partition* carries the information and the label does not.

What counts as agreement is the ontology's judgment, not ours. The relation above is inherited wholesale from the Cell Ontology: two calls agree when CL says one subsumes the other. Where CL is coarse or incomplete, two references naming the same population can be scored as

disagreeing; where it is deep, a pair of calls can agree at a resolution neither reference meant to assert. The comparison was possible at all only because Pan-human Azimuth publishes a CL term for every node of its typology; an annotator with no crosswalk cannot enter this analysis regardless of how good its labels are.

The alternative design targets exactly this problem, and looking at it closely shows why the substitution is not free. The Common Cell type Nomenclature [Miller 2020] gives each taxonomy's cell sets stable accessions (`CS[taxonomy id]_[n]`) plus a curated alias layer, treating CL as one alignment target rather than as the coordinate system. In the published human middle temporal gyrus taxonomy `CCN201908210`, 154 cell sets carry an accession and a structure tag — `UBERON:0002771`, the gyrus itself — but **no cell-type ontology id at all**; the field that matches cell sets across taxonomies, the aligned alias, is populated for **23 of the 154**. Of those 23, eight match a CL term name or synonym by exact string (`L2/3 IT`, `L5 IT`, `L5/6 NP`, `L6 CT`, `L6 IT`, `Lamp5`, `OPC`, `Sst Chodl`); most of the rest are abbreviations such as `Astro`, `Oligo` and `Endo` whose expansions do exist in CL, so the true overlap is larger than a string test finds and is exactly as large as someone is willing to curate.

The 151-type brain split of §3.1 uses those cell sets as its label set. Because they carry no ontology id, calls at that resolution can only be checked against the taxonomy's own strings: that split is scored by exact match alone, and the ranking it produces — correlation methods first, trained classifiers last — cannot be cross-checked against the ontology-aware column used elsewhere in this paper.

That is the substantive difference. CL supplies a *total* subsumption relation — every pair of terms is comparable, at whatever granularity CL happens to encode — which is what makes an agreement partition computable at all. CCN supplies exact provenance, which CL cannot: it records which taxonomy, which algorithm and which publication a label came from. An agreement relation built on CCN accessions alone would be undefined for most pairs of cell sets, because accessions are deliberately taxonomy-scoped rather than shared. The two systems answer different questions, and a workflow that compares annotations across references needs both: CL to decide whether two calls are compatible, CCN to say what each call actually was.

3.5 Rejection / abstain

Holding out 9 of 36 cell types entirely from the HLiCA liver reference (so 1,350 query cells are genuinely out-of-distribution), we sweep a confidence threshold over every method that returns a per-cell confidence. Eight do; for scANVI and scArches the confidence is the maximum class posterior from the model's soft prediction. SVM, SingleR and scPRINT return no per-cell confidence and cannot be swept, and scmap-cluster has a single native "unassigned" decision rather than a threshold.

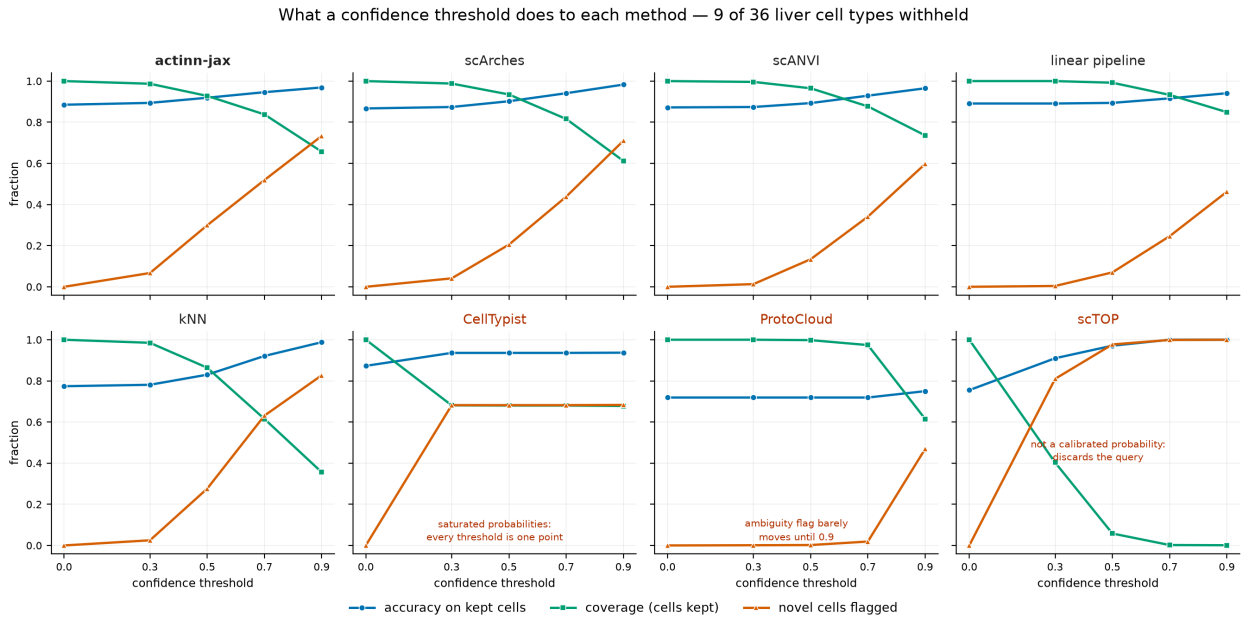

Figure 7. What a confidence threshold does to each of the eight methods that return a per-cell confidence, with 9 of 36 cell types held out of the liver reference so 1,350 query cells are out-of-distribution. Every quantity is a fraction, so all three share one axis: accuracy on kept cells, coverage, and the share of held-out-type cells flagged. The five that work share a shape — accuracy and novelty rising as coverage falls — and the three that do not each fail visibly and differently. Per-threshold values are in `results_rejection.csv`.

Five of the eight give a threshold that does something; three do not. actinn-jax, scArches, scANVI, the linear pipeline and kNN all trade coverage for accuracy across the range. The other three fail in different ways: CellTypist's probabilities are saturated near 0 or 1, so every threshold from 0.3 to 0.9 lands on one operating point; scTOP's projection score is not a calibrated probability and discards all but 6% of the query by $p \geq 0.5$; ProtoCloud's ambiguity flag barely moves until 0.9.

Among the five that work, abstain quality does not separate them the way cost does (Figure 8). actinn-jax and scArches are effectively tied — at $p \geq 0.9$, 0.969 accuracy on 66% of cells with 73% of novel cells flagged against 0.983 on 61% with 71% flagged — and scANVI is close behind. kNN reaches the highest novelty detection (83%) by keeping only 36% of the query, which is a different operating regime rather than a better one, and the linear pipeline is the weakest here, flagging 46% of novel cells at the threshold where actinn-jax flags 73%. The distinction worth drawing is therefore not that actinn-jax abstains better than the deep methods but that it abstains as well for 0.54 s of predict time against scArches's 17.2 s and scANVI's 66.7 s (Table 3) — the same accuracy-at-a-fraction-of-the-cost pattern §3.1 reports, applied to the mechanism the workflow of §3.4 routes on.

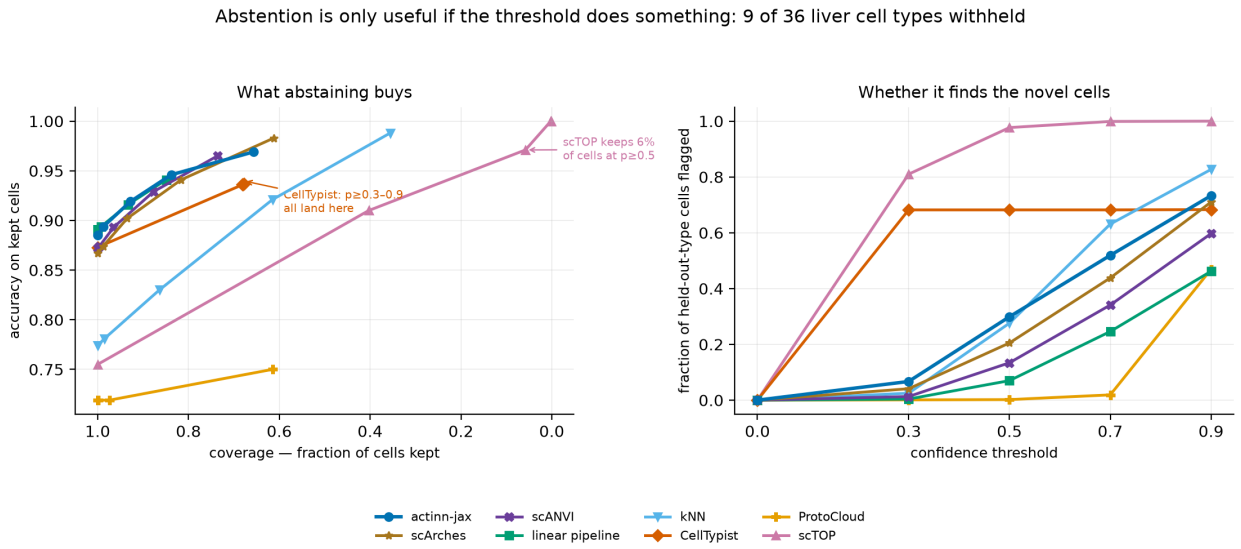

Figure 8. The same sweep read as a trade-off, which is what compares methods to each other rather than to themselves. *Left:* accuracy on kept cells against the fraction kept — the five usable methods lie along one band, which is the basis for calling their abstain quality tied. *Right:* the share of held-out-type cells flagged as the threshold rises. The three methods Figure 7 shows failing appear here as curves that go nowhere along one axis or the other.

3.6 Foundation-model zero-shot (scPRINT)

scPRINT is run separately as a zero-shot predictor (no training on the reference), as a reference point for the "just use a foundation model" alternative. On the lung dataset it scored **exact-match accuracy 0.029 / ontology concordance 0.201, taking 62 s for 2,694 query cells** — against actinn-jax's 0.894 in 0.23 s — so as a *label* predictor it is both slow (~**280**× actinn-jax) and weak. It is reported on lung alone because its fixed vocabulary cannot score the other datasets: the liver and blood+gut sets contain Cell Ontology terms absent from its label set, and it declines them rather than guessing. That is an inherent property of a zero-shot label head rather than a defect, but it does mean the raw predictions are not a drop-in annotator. It is why the two-stage workflow uses scPRINT's **embeddings** — its learned structure — to shape a small trained model, and never its zero-shot labels.

3.7 External validation: Open Problems `label_projection`

Our in-house benchmark is neutral but self-run. For an **independent** check, we ran actinn-jax through the community-standard Open Problems label-projection task — their 6 datasets, splits, and metrics — and slotted it into their published v2.0.0 leaderboard.

Placement. On a benchmark we did not design and cannot tune, **actinn-jax places 3rd of 17 on mean accuracy (0.837), 1st among all methods that complete every dataset**, beats its PCA-space `mlp` sibling, and posts the best accuracy of any completing method on the hardest dataset (tabula_sapiens, 160 types: 0.394 vs mlp 0.342). It is upper-mid on macro-F1 and **not the top method overall** — scANVI+scArches surgery and xgboost [Chen & Guestrin 2016] score higher on the datasets they finish, though both **fail to complete tabula_sapiens**, which is why they rank above actinn-jax only on a mean over the 5 easier datasets. The foundation models again

land at the bottom (scgpt_zeroshot 0.639, uce 0.131 ≈ the random-labels control), reproducing §3.6 externally.

Means over the six datasets, every method through the same pipeline on one instance:

method	mean acc	macro-F1	cost	peak mem
mlp	0.843	0.662	1.98×	19.9 GB
actinn-jax	0.836	0.663	1.00×	21.0 GB
linear-anova-pca	0.828	0.647	2.67×	20.1 GB
SVM (SGD)	0.816	0.652	6.07×	20.1 GB
logistic_regression	0.813	**0.689**	**0.18**×	20.0 GB
CellTypist	0.810	0.643	7.62×	20.1 GB
knn	0.793	0.648	**0.07**×	19.5 GB
xgboost	0.791	0.614	5.48×	80.7 GB
cellmapper_linear	0.776	0.553	0.35×	31.5 GB
naive_bayes	0.738	0.613	0.19×	19.5 GB
scTOP	0.581	0.462	**0.16**×	20.4 GB

Table 9. All eleven methods on the six Open Problems datasets, ordered by accuracy. Bold marks the five components we contributed. *cost* is per-dataset wall-clock relative to actinn-jax (§2.10), for which 1.00× is roughly two minutes.

actinn-jax **matches its `mlp` sibling on accuracy at half the cost**, and **beats xgboost on accuracy at 5.5× less runtime and 4× less memory** (21 vs 81 GB). Everything cheaper — knn, logistic_regression, naive_bayes, scTOP — is less accurate, and the one heavyweight is slower and heavier. Nothing here beats it on accuracy and cost together.

The tuned linear pipeline loses its §3.1 advantage on both axes. It places third on accuracy (0.828, behind `mlp` and actinn-jax), and where it fits 7× *faster* than actinn-jax on the in-house panel it costs **2.7× more** here. The input budget explains the reversal: OP hands every method 1,000 HVGs, and an ANOVA→PCA→logistic pipeline pays for the decomposition on every query while a gene-space MLP amortizes it into one fit. CellTypist (7.6×) and the SGD-SVM (6.1×) shift the same way. Neither a cost ranking nor an accuracy ranking survives a change of feature budget — which is why we report both panels rather than picking one.

`logistic_regression` places fifth on accuracy but first on macro-F1 (0.689), at the second-lowest cost. Across datasets spanning 13 to 160 cell types, accuracy and macro-F1 do not order the panel the same way.

scTOP collapses on two of the six and is weak on a third. Its mean (0.581) is not a uniformly weak result but 0.042 on tabula_sapiens, 0.124 on gtex_v9 and 0.495 on immune_cell_atlas against 0.90–0.99 on the other three. Open Problems hands every method 1,000 HVGs, and scTOP's rank projection needs genes expressed in an appreciable fraction of cells; after its expression filter only **106 of 1,000** genes survive on gtex_v9 and 210 on tabula_sapiens, against 306–405 where it works. At that point some test cells retain no counts at all — 185 of 11,508 on gtex_v9 — and cannot be placed; they are scored as wrong rather than dropped, so 0.124 is a floor. This is the same limitation §3.1 reports from the other direction: scTOP is built for small, low-cardinality problems, and a fixed 1,000-HVG budget over 53 and 160 types is neither.

Three cheap ablations, and their limits. (i) *Input standardization* — z-scoring genes to the reference's frozen mean/std, a CPU-only domain-alignment inspired by scArches reference surgery — lifts mean accuracy +0.2 and macro-F1 +1.2 pt (largest on batch-shifted datasets: gtex macro-F1 +7.9). It is **off by default** because it shifts the softmax calibration that the two-stage abstain thresholds are tuned against. (ii) *Gene budget* — OP feeds every method 1000 HVGs, which starves a gene-space MLP more than it starves the others. Widening to ~5000 HVGs lifts accuracy on 4 of 6 datasets (immune/gtex +3 pt) and there **matches scANVI+scArches's full-config leaderboard accuracy** (immune 0.891 ≈ 0.892) on CPU in minutes — the gap to the top method was largely a gene-budget artifact, not the VAE. But more genes is **not** a universal win: it *regresses* tabula_sapiens by ~10 pt (its 284-cell test batch across 160 fine types overfits reference-specific genes) and saturates hypomap (Figure 9). The budget is **selectable without test labels**: held-out *reference* cross-validation rises for the datasets that benefit and is the one signal that *drops* for tabula_sapiens, and a trivial query-cells-per-class check independently flags it (Figure 10) — so the budget can be set deterministically per dataset. (iii) *Negative control* — a CPU-only, UCE-style protein-embedding featurization (expression-weighted mean of ESM2 gene embeddings) does **not** help and hurts the hardest case: the pooling discards the per-gene detail that separates fine types, so the value of a foundation model like UCE stays locked in its GPU transformer, not a portable averaging trick.

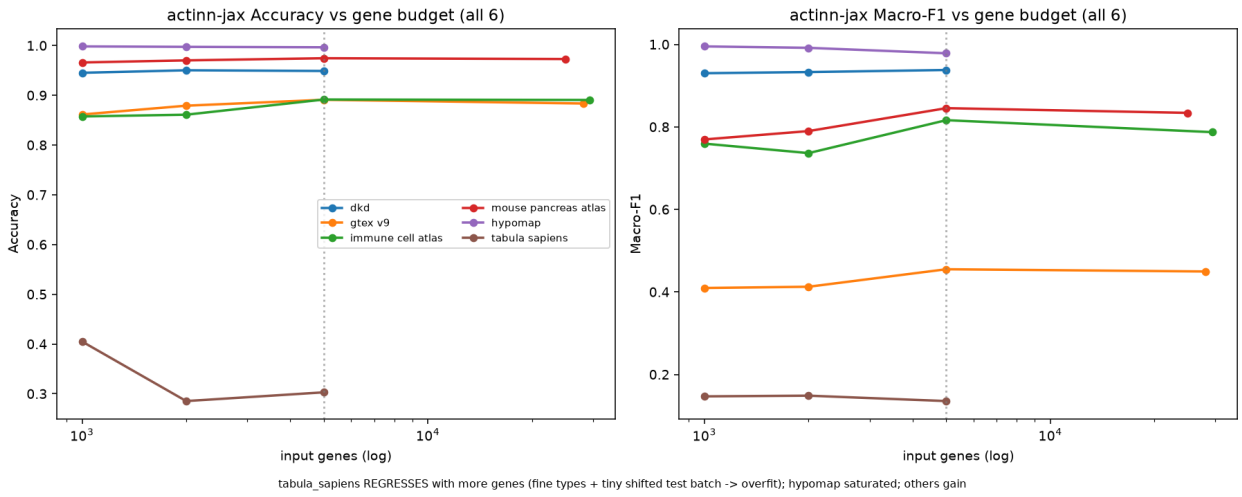

Figure 9. actinn-jax accuracy and macro-F1 against input gene budget across all six Open Problems datasets. More genes help most datasets but regress the fine-grained, domain-shifted tabula_sapiens.

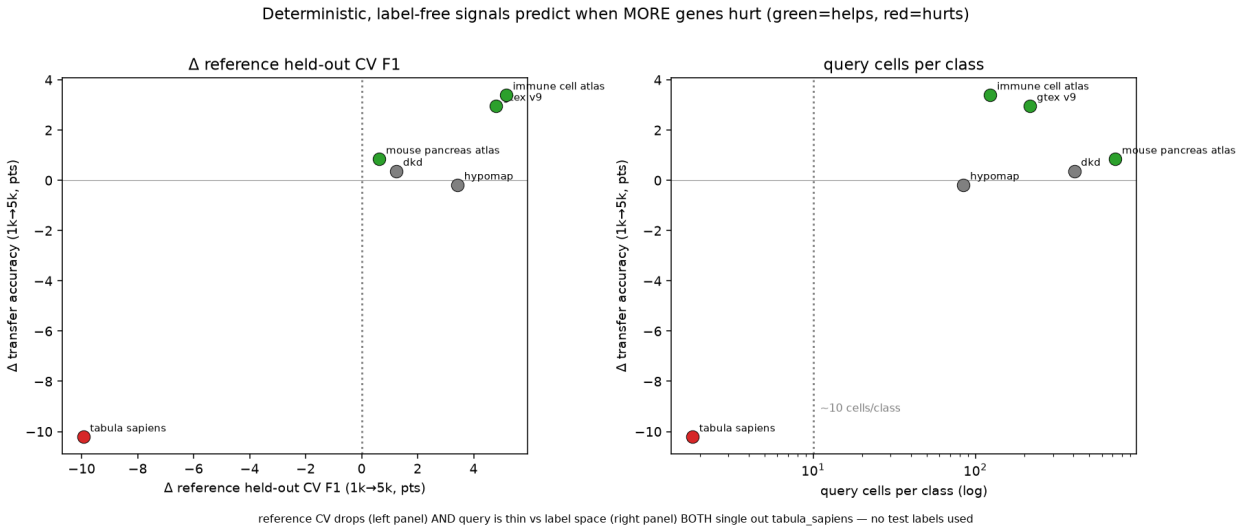

Figure 10. Label-free signals for setting the gene budget without test labels. Held-out reference cross-validation and query-cells-per-class both single out tabula_sapiens, the one dataset where more genes cost real accuracy — about 10 points. hypomap drifts down as well, but from a saturated 0.998, and neither signal flags it.

4 Discussion

Method choice is now dominated by cost, not accuracy. The four leading methods are separated by 0.008 in mean accuracy and by ~205× in predict time (§3.1), so for most annotation jobs the decision is made on the cost axes. On laptop-sized references the tuned linear pipeline is the best of them on accuracy-per-second (0.839, fitting in 3.0 s); at atlas scale ProtoCloud is the most accurate by a clear margin (0.976 vs 0.936 at 49k lung cells; 0.905 vs 0.824 at 47k liver cells), having been below the cluster on the subsampled matrix. actinn-jax's place in that picture rests on a cost profile with two properties that matter for repeated use rather than for a single labeling run.

Footprint. Inference is **sub-second** — flat in reference size, and rising 3.2× with cardinality while staying under a third of a second (§3.3) — so a query costs the same against a 1k-cell reference and a 49k-cell one. Peak memory is unremarkable on small references (2.4 GB mean — seven of the ten other methods are lighter), but it is the lowest of those we carried to **atlas scale**: ~2× below the linear pipeline (6.1 vs 13.2 GB at 49k lung cells; 6.5 vs 12.6 GB at 47k liver cells), and below scTOP above ~25k cells, where scTOP's rank processing densifies and crosses over (9.3 vs 6.5 GB at 47k). The advantage is bounded at ~2–3× rather than widening. Combined with the train-once/map-many cache, a reused reference pays fit once and then only the flat predict cost, where the linear pipeline refits scaler/PCA/classifier for every query.

Those two properties are what the workflows are built on. The practical gains come not from the flat classifier but from the **broad→focused** pipeline (§3.4): a census-scale broad model with a calibrated abstain routes a query to a small focused reference that re-annotates at full resolution (cross-study liver 0.23/0.58 → 0.72/0.86), with zonation as a further stage. The pipeline is only usable because each hop is cheap — a flat sub-second call against a cached model, at bounded memory — so several models can be chained, and a large reference kept resident, on a laptop without a GPU. A method that is a few points more accurate but pays a full fit per query, or a

multi-gigabyte dense expansion per stage, does not compose the same way.

Where it loses, and why. Much of what looked like a deficit of the *model* on Open Problems was a deficit of its **input**. The benchmark hands every method 1,000 highly variable genes, a far harsher constraint on a model reading gene space directly than on one that learns a compressed representation first, and at a fair gene budget the gene-space MLP *matches the leaderboard's top method* on the datasets both complete. Widen the gene panel and most of the gap closes, so the quantity being measured was partly "how well does each method tolerate a narrow input", not "which model class is stronger".

The genuine weaknesses are narrower. actinn-jax trails on small references and very-few-cells-per-type sets (§3.1, §5), and it is sensitive to input budget under domain shift: more genes help most datasets but overfit a tiny, fine-grained, shifted query (tabula_sapiens). The two cheapest levers are principled and label-free — **standardization** (a scArches-style domain alignment) and a **gene budget chosen by cross-validation on the reference alone** — and neither peeks at test labels.

Foundation models. Their **zero-shot labels** are the weakest option in both benchmarks (scPRINT, scGPT, UCE ≈ random); their **embeddings/structure** are where the value lies, and our two-stage hierarchy uses exactly that. A cheap CPU shortcut to a foundation-model representation (protein-embedding pooling) did *not* transfer (§3.7), underscoring that the value is in the trained transformer, not a portable trick. The concurrent **Pan-human Azimuth** effort reaches the same conclusion from a far larger curated corpus. Their 7M-parameter supervised hierarchical MLP over a 5,055-gene panel yields cleaner annotations than scGPT and SCimilarity; they report those models' labels as fragmented and mis-transferred. Their stated theme — that training-data quality and organization matter as much as architecture or scale — is one of the two claims we draw here, and their finding that accuracy **saturates past ~5M training cells** supports it further. The other claim, that zero-shot foundation-model labels are the weaker option, they reach independently. Both come from a group with no stake in our conclusion.

When extra expressivity pays. Two lines of evidence bear on this. First, **ProtoCloud** — a prototype-based, self-explaining VAE with built-in uncertainty and gene-level attribution, a strictly richer model than ours — does not sit a fixed distance above or below us. Where it lands depends on how much reference data it is given: below the top cluster on the subsampled matrix, ahead on lung, and the strongest method of all at atlas scale (§3.1, §3.3), at ~8× actinn-jax's CPU fit time (176 vs 22 s mean). The richer model pays off where it has the data to support it, and not before — which is an argument for matching model capacity to reference size, not for preferring small models everywhere. Second, and more broadly, Souza & Mehta [Souza & Mehta 2026] report that **parameter-free linear representations** match or exceed single-cell foundation models across cross-species transfer, human cell-type classification and disease-state prediction. On Tabula Sapiens 2.0 their normalize→ANOVA→PCA→logistic-regression pipeline reaches mean macro-F1 0.899, against 0.907–0.910 for TranscriptFormer variants. Their explanation is geometric: the biologically realized cell manifold is well approximated by a **linear subspace**, so once noise is suppressed performance *saturates* and extra expressivity buys little. That is a principled account of why a four-layer MLP over normalized gene space stays competitive, and it matches our own finding that the residual gap to heavier methods was mostly **the size of the input each method was given rather than the class of model** (§3.7). They further find that simple methods hold up *best* out-of-distribution — on novel cell types and unseen organisms. That matters for the abstain mechanism of §3.5, which is only useful if low confidence tracks genuine novelty rather than model noise. A method that degrades gradually on unfamiliar data still assigns lower confidence to the cells it is getting wrong, so a threshold separates them; a method that degrades abruptly can be

confidently wrong, and no threshold helps.

Practical guidance. For a **one-off annotation** on laptop-sized data, use the **tuned linear pipeline** (normalize → ANOVA → PCA → logistic regression): it is the most accurate-per-second method here. With a **real atlas and time to train**, **ProtoCloud** is the most accurate method we benchmarked (0.976 at 49k cells), and the GPU it targets makes its 19× CPU fit cost much less punishing in practice. **scTOP** suits small, low-cardinality problems, where it is by far the cheapest (~1 s) and competitive. **actinn-jax** fits the cases its cost profile is shaped for: when **memory is the binding constraint** (~2× lighter than the linear pipeline at scale, with memory-bounded chunked inference), when a reference is **reused** across many queries (the cached model amortizes the fit), and when the job is the **multi-stage workflow** rather than a single label — broad→focused hand-off, tissue-aware refinement, calibrated abstain, novel-cell-type screening.

For the **broad pass specifically, prefer a purpose-built pan-human model**: **Pan-human Azimuth** is a pretrained hierarchical classifier over a harmonized organism-wide typology (8 levels, 382 leaf types, trained on 9.7M curated cells), with abstention learned rather than thresholded and an **expected calibration error (ECE)** of 0.0044 — meaning its stated confidence sits within half a percentage point of its observed accuracy — running at ~1,000 cells/s on a laptop. It is better resourced than our census-built reference and we make no claim to improve on its annotations. What we do with it instead is **use it**: distilling its labels and its hierarchy into actinn-jax produces a broad-pass model of comparable accuracy at six to nine times its throughput, built without a GPU and without labeled data (§3.4). A curated pan-human model is the right thing to start from; a small fast model is the right thing to iterate with.

What no fixed typology can do is the **hand-off**: it cannot re-annotate into a user's own focused label set (the HLiCA liver reference, cross-study 0.23/0.58 → 0.72/0.86) or resolve states below a leaf (hepatocyte zonation). ProtoCloud likewise provides uncertainty, gene attribution and a refinement path (fine-tuning a pretrained model). What survives as distinct to actinn-jax is therefore not the breadth of the broad reference but **the chain it enables** — start pan-human, re-annotate into label sets no pretrained model carries, screen what none of them claims — at a cost that keeps every stage on a laptop. The distillation recipe is not specific to this teacher or to humans: it needs a pretrained annotator whose typology can be read off its outputs, so extending the same entry point to other organisms is a matter of finding one (or building the broad-pass reference from a labeled census slice, as §3.4's census route does).

What the speed is for. Annotation is almost never the result; it is the step before the biology, and it is usually run more than once — the labels change when a reference is swapped, a threshold moves, a new sample arrives, or a cluster turns out to be two things. The argument of this paper is not that a faster classifier produces better labels; at equal data the leading methods differ by at most 0.008 in accuracy (§3.1). It is that when a stage costs a second instead of a minute, and a chain of stages fits in memory on the machine already on the desk, the loop from question to annotated data to the next question closes in an afternoon rather than across a queue. That is the contribution we would defend: not a better number in a table, but a workflow — start from the best pan-human model available, refine into the labels a given study actually uses, and flag what neither covers — that a working scientist can run, inspect, and run again.

5 Limitations

- Single hardware family for the in-house panel (Apple Silicon); no discrete-GPU numbers there. The deep and foundation methods are written for CUDA and we would expect them to be faster on it, but we did not measure that — it is outside the "runs-on-a-laptop" question, and every timing here should be read as this hardware's. The §3.7 controlled run covers the **CPU tier** on one cloud box; the GPU/R methods there are reported from OP's own cloud-CI trace (indicative). A same-hardware GPU-tier run to fold those into a single controlled table is the natural extension.
- **Wall-clock comparisons across methods are only meaningful at matched concurrency.** Measured `%cpu` on the OP harness spans 49% to 2338% across methods — some are single-threaded, some saturate 23 cores — so the same method timed at two concurrency settings differs by up to 12×, and any wall-clock ranking taken at high concurrency silently ranks threading and scheduler contention alongside algorithm. Table 9 therefore reports cost as a ratio within a run, anchored by a method common to both (§2.10) — the anchor itself moved 165 s → 87 s between them, which is the size of the effect.
- **The cross-method comparison is human only**; eight splits per benchmark; GPU foundation models beyond scPRINT/UCE (scGPT, Geneformer, popV) not run locally. The bundled references now cover mouse (§3.4), but no *method comparison* was run on mouse data — the pan-mouse result establishes that the reference-building route works on a second organism, not that actinn-jax's standing against other methods carries over to it.
- **actinn-jax's standing is worse at very fine granularity.** On the 151-type brain cluster split it places tenth of eleven (0.741 against SingleR's 0.845, §3.1), and the two correlation methods that trail everywhere else lead there. The panel covers one split at that cardinality, and it runs opposite to the 86-type blood+gut result; Table 3's aggregate does not cover annotation at cluster resolution.
- actinn-jax needs more cells/type than linear methods to reach parity (§3.1); on very small references it trails.
- **The distilled broad-pass reference inherits a vocabulary, not a calibration.** Pan-human Azimuth's abstention is trained (and its ECE measured at 0.0044); the distilled student learns hard labels only, so it carries none of that — its confidence barely separates right from wrong (§3.4). Recalibrating it, or distilling from soft targets, is the obvious next step and would need a loss change in the package. Its evaluation is also two withheld *atlases*, not withheld tissue: the census-wide corpus spans 376 tissues, so nothing here measures behavior on biology the corpus never saw.
- **The gene budget is dataset-dependent, not a free parameter.** More genes help most references but *overfit* a tiny, fine-grained, domain-shifted query (tabula_sapiens −10 pt); our reference-cross-validation + cells-per-class selection rule is validated on 6 datasets with a single clean failure case, so it is directional evidence, not a tuned threshold.
- **Standardization is opt-in**, not default, because it shifts probability calibration that the two-stage abstain thresholds are tuned against; combining the two cleanly would require re-tuning those thresholds.
- **Our classical tier (SVM, kNN, CellTypist) is untuned, and a tuned linear baseline beats actinn-jax.** Souza & Mehta's pipeline (normalize → ANOVA → standardize → PCA(220) → logistic regression) leads the matrix on accuracy (0.839 vs 0.831) and macro-F1 (0.699 vs 0.683) while fitting 7× faster on this panel's hardware (§2.5), with its largest margin on the 86-type blood+gut set (+4.2 pt). Wall-clock ratios between a JAX model and a scikit-learn pipeline shift with the CPU and the threading available, so read them as this

machine's, not as constants. The §3.1 margins over the classical tier should therefore be read as margins over *untuned* baselines; a like-for-like tuning effort on SVM or CellTypist would likely narrow them further. scTOP is cheaper still to fit but competitive only at small, low cardinality — its own best macro-F1 on pbmc (0.837), degrading to 0.644 on liver.

- **actinn-jax's memory advantage is bounded, and the linear pipeline scales further than a naive extrapolation implies**. Extrapolating the linear pipeline's small-reference footprint overstates its atlas-scale peak by ~2.4× — the measured figure is **13.2 GB** at 49k reference cells — because its feature-selection step caps the dense matrix at 20,000 genes rather than the full gene set. The linear/actinn-jax memory ratio is likewise **bounded at ~2–3×** (2.15× at full scale) rather than widening with data, so there is no scaling cliff and no regime tested where the linear pipeline stops being usable. Memory in this setting should be measured, not extrapolated.
- **The main matrix uses subsampled references, and its ranking does not survive atlas scale.** Carried to 49k lung and 47k liver reference cells, ProtoCloud moves from the weakest method to the strongest and scTOP crosses from the lightest to heavier than actinn-jax (§3.3, Figure 3). The accuracy ordering of Table 3 describes laptop-sized references, which is the regime this paper is about, but it should not be read as a ranking at atlas scale.
- **Annotation only.** Cross-species transfer and disease-state prediction — the tasks where Souza & Mehta find the largest foundation-model deficits, and where a fast method would be most attractive — are outside this benchmark's scope (human, within/cross-dataset annotation).

Data & code availability

Two repositories hold everything needed to reproduce this work. The **benchmark repository** carries the harness, the per-method adapters, the run configurations that produce every number reported here, the environment lockfiles, the result tables including the unified eleven-method matrix, and the figure scripts. It is archived at doi:10.5281/zenodo.21911372 (MIT; a concept DOI that resolves to the most recent release), which is what to cite for reproduction rather than the default branch, since the branch may move on. The **actinn-jax package** is on PyPI (`pip install actinn-jax`). Rebuilding the broad reference is a single documented command; the distilled broad-pass reference, the pan-mouse reference and the Cell-Ontology hierarchy it depends on each have their own build documentation in the benchmark repository, and the file-level detail lives there rather than here.

Pre-trained references — human (`broad_human_v1`, `panhuman_distill_v1`), mouse (`broad_mouse_v1`) and focused liver (`liver_hlica_v1/v2`) — are archived at doi:10.5281/zenodo.21688151 (CC BY 4.0; cite the concept DOI 10.5281/zenodo.21688150 for the latest version). The package downloads and caches them on first use, so no manual retrieval is needed, and can pre-download them for offline use.

HLiCA data © Edgar et al. 2026 (CC-BY 4.0), [doi:10.64898/2026.06.30.735539]. The distilled reference derives from **Pan-human Azimuth** (Sarkar, Li, Molla, … Satija, bioRxiv 2026, doi:10.64898/2026.07.16.738997); its weights are © the authors under CC BY 4.0, obtained via `panhumanpy` (MIT) and Zenodo. Attribution is a condition of that licence and travels inside each model's build record.

Additional documentation. The benchmark repository carries detailed notes for each result — tuned linear pipeline and sctop baselines; protocloud comparison; scaling and memory to atlas

size; survey of cell-type annotation methods; rebuilding the broad reference; distilling pan-human azimuth; and 9 more.

Supplementary material (separate document) contains Figures S1–S11 — confusion matrices with ontology-equivalent errors outlined, per-class recall across eleven methods on four splits, the cost and scaling studies, the abstain sweeps, and the per-split view behind Figure 1 — and Tables S1–S3, the method and dataset descriptions and per-dataset accuracy.

References

1. 10x Genomics. 3k PBMCs from a healthy donor, Cell Ranger 1.1.0 (2016). Distributed with scanpy as `pbmc3k`. 10xgenomics.com/datasets/3-k-pbm-cs-from-a-healthy-donor-1-standard-1-1-0.
2. Abdelaal T, et al. A comparison of automatic cell identification methods for single-cell RNA sequencing data. *Genome Biology* 20:194 (2019). doi:10.1186/s13059-019-1795-z.
3. Alegbe T, Harris BT, Fachal L, et al. Cell-type-resolved genetic variation shapes inflammatory bowel disease risk (IBDverse). *Nature* (2026). doi:10.1038/s41586-026-10627-z.
4. Aran D, et al. Reference-based analysis of lung single-cell sequencing reveals a transitional profibrotic macrophage (SingleR). *Nature Immunology* 20:163-172 (2019). doi:10.1038/s41590-018-0276-y.
5. Bradbury J, et al. JAX: composable transformations of Python+NumPy programs (2018). github.com/jax-ml/jax.
6. CZI Cell Science Program. CZ CELLxGENE Discover Census, LTS release 2025-11-08. chanzuckerberg.github.io/cellxgene-census.
7. Chen T, Guestrin C. XGBoost: a scalable tree boosting system. *KDD* 785-794 (2016). doi:10.1145/2939672.2939785.
8. Domínguez Conde C, et al. Cross-tissue immune cell analysis reveals tissue-specific features in humans (CellTypist). *Science* 376:eabl5197 (2022). doi:10.1126/science.abl5197.
9. Edgar RD, Portman JR, Hu H, et al. HLiCA: an integrated cell atlas of the healthy human liver. *bioRxiv* (2026). doi:10.64898/2026.06.30.735539.
10. Fu Q, Dong C, Liu Y, et al. A comparison of scRNA-seq annotation methods based on experimentally labeled immune cell subtype dataset. *Briefings in Bioinformatics* 25(5):bbae392 (2024). doi:10.1093/bib/bbae392.
11. Guo K, Ding J. ProtoCloud: a prototypical self-explaining model for single-cell analysis. *Cell Genomics* 6(6):101217 (2026). doi:10.1016/j.xgen.2026.101217.
12. Jorstad NL, Song JHT, Exposito-Alonso D, et al. Comparative transcriptomics reveals human-specific cortical features. *Science* 382(6667):eade9516 (2023). doi:10.1126/science.ade9516.
13. Kalfon J, Samaran J, Peyré G, Cantini L. scPRINT: pre-training on 50 million cells allows robust gene network predictions. *Nature Communications* 16:3607 (2025). doi:10.1038/s41467-025-58699-1.
14. Kedzierska KZ, Crawford L, Amini AP, Lu AX. Zero-shot evaluation reveals limitations of single-cell foundation models. *Genome Biology* 26:101 (2025). doi:10.1186/s13059-025-03574-x.
15. Kiselev VY, Yiu A, Hemberg M. scmap: projection of single-cell RNA-seq data across data sets. *Nature Methods* 15:359-362 (2018). doi:10.1038/nmeth.4644.
16. Lin Z, et al. Evolutionary-scale prediction of atomic-level protein structure with a language model (ESM-2). *Science* 379:1123-1130 (2023). doi:10.1126/science.ade2574.
17. Lotfollahi M, et al. Mapping single-cell data to reference atlases by transfer learning

(scArches). *Nature Biotechnology* 40:121-130 (2022). doi:10.1038/s41587-021-01001-7.
18. Ma F, Pellegrini M. ACTINN: automated identification of cell types in single cell RNA sequencing. *Bioinformatics* 36(2):533-538 (2020). doi:10.1093/bioinformatics/btz592.
19. Miller JA, Gouwens NW, Tasic B, et al. Common cell type nomenclature for the mammalian brain. *eLife* 9:e59928 (2020). doi:10.7554/eLife.59928.
20. Open Problems for Single-Cell Analysis Consortium. Open Problems: a living benchmark for single-cell analysis (2024). openproblems.bio.
21. Pedregosa F, et al. Scikit-learn: machine learning in Python. *JMLR* 12:2825-2830 (2011). jmlr.org/papers/v12/pedregosa11a.html.
22. Rosen Y, et al. Universal cell embedding provides a foundation model for cell biology (UCE). *Nature* (2026). doi:10.1038/s41586-026-10689-z.
23. Sarkar S, Li Z, Molla G, et al. Organism-scale annotation with Pan-human Azimuth. *bioRxiv* (2026). doi:10.64898/2026.07.16.738997.
24. Sikkema L, et al. An integrated cell atlas of the lung in health and disease (HLCA). *Nature Medicine* 29:1563-1577 (2023). doi:10.1038/s41591-023-02327-2.
25. Souza H, Mehta P. Parameter-free representations outperform single-cell foundation models on downstream benchmarks. *bioRxiv* (2026). doi:10.64898/2026.02.11.705358.
26. Travaglini KJ, Nabhan AN, Penland L, et al. A molecular cell atlas of the human lung from single-cell RNA sequencing. *Nature* 587(7835):619-625 (2020). doi:10.1038/s41586-020-2922-4.
27. Xu C, et al. Probabilistic harmonization and annotation of single-cell transcriptomics data with deep generative models (scANVI). *Molecular Systems Biology* 17:e9620 (2021). doi:10.15252/msb.20209620.
28. Yampolskaya M, Herriges MJ, Ikonomou L, Kotton DN, Mehta P. scTOP: physics-inspired order parameters for cellular identification and visualization. *Development* 150(21):dev201873 (2023). doi:10.1242/dev.201873.
29. Munroe R. Standards. *xkcd* 927. xkcd.com/927.